

ES 336: Computer Organization and Architecture



Sameer G Kulkarni

Assistant Professor,

*Department of Computer Science and Engineering,
Indian Institute of Technology, Gandhinagar*

Today's Lecture

- Memory Design

- Memory Design Aspects
- Memory Hierarchy
- ~~Virtual Memory~~

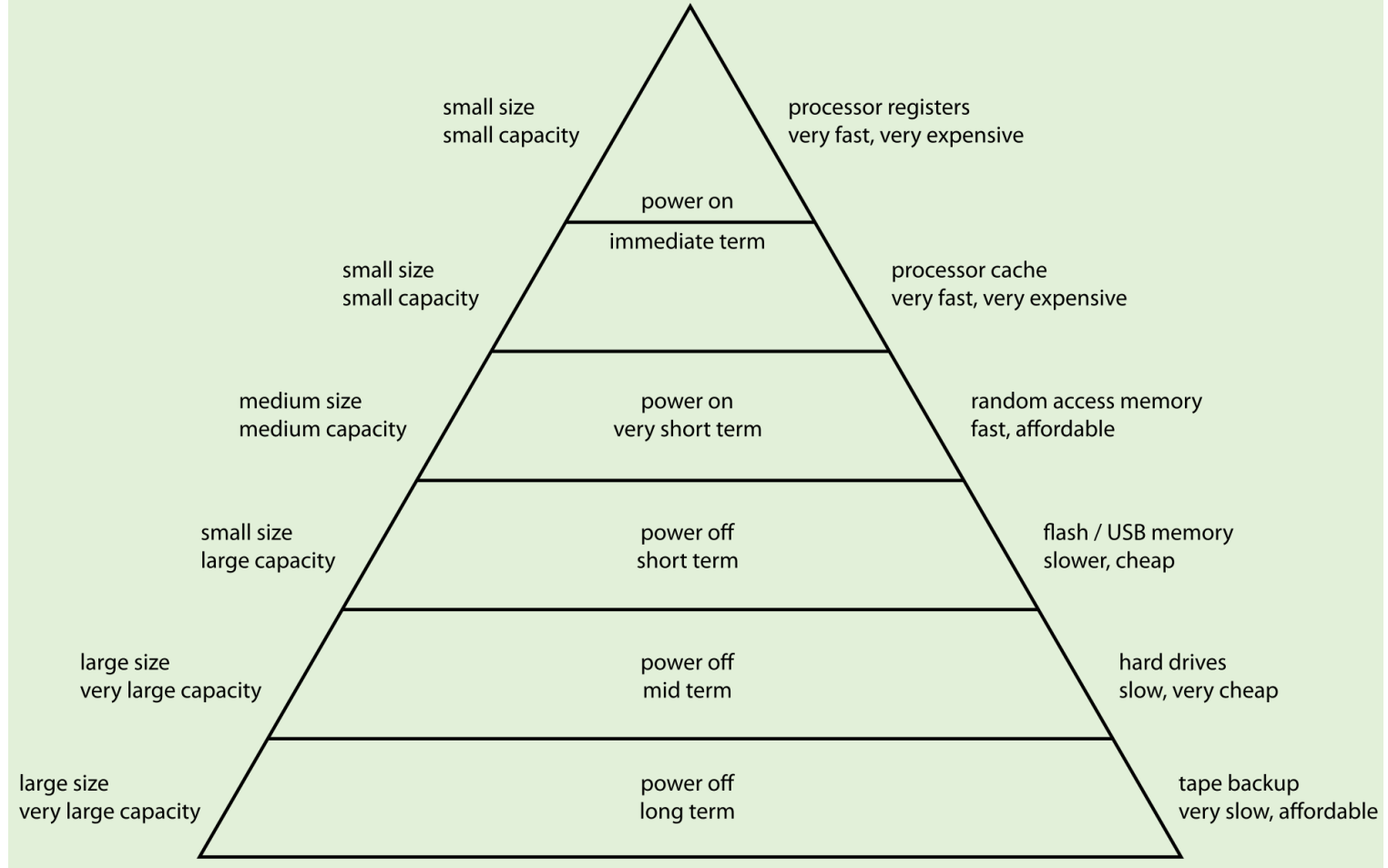
- Key Research Topics

- Optimizations (techniques)
- Memory Access Paradigms
- Memory technologies

- Reading Materials:

- Chapter 5 – COAHP
- Chapters 4,5 – COAWS

Computer Memory Hierarchy



Source: [Wikipedia](#)

INSTRUCTIONS & OPERANDS

- A computer needs a physical location from which to retrieve the Instructions & operands

- A computer retrieves operands from:

- Registers
- Constants (also called immediate)
- Memory

- Main Memory is slow – Registers for Faster Access.

- Most architectures have a small set of (fast) registers

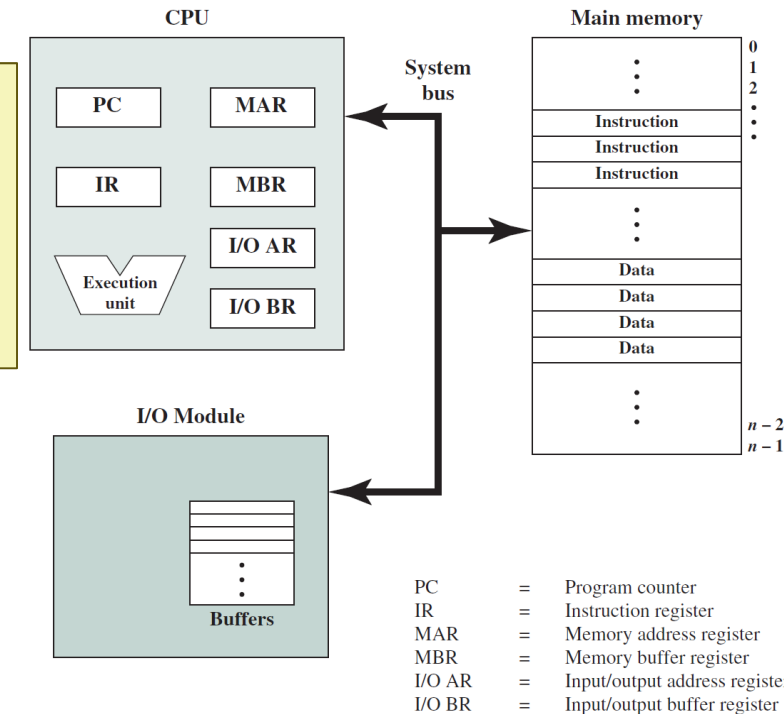
- MIPS/DLX have *thirty-two* 32-bit registers.

Name	Example	Comments
32 registers	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants.

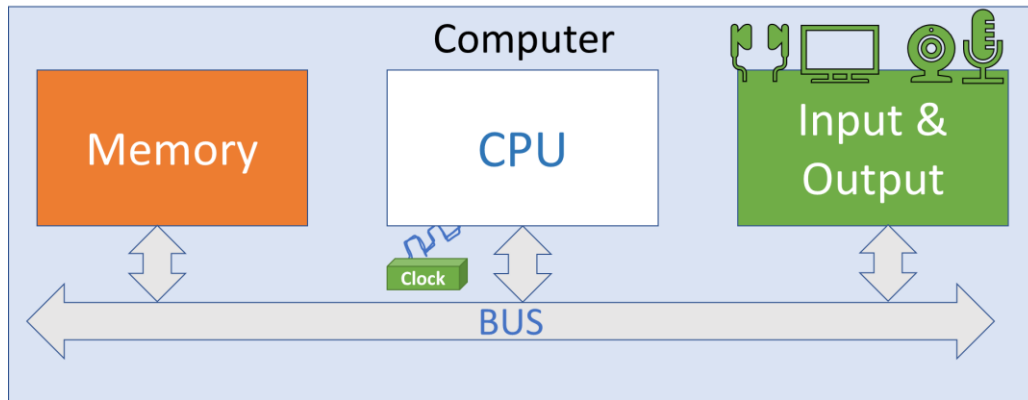
- MIPS is called a 32-bit architecture because it operates on 32-bit data

- A 64-bit version of MIPS also exists, but we will consider only the 32-bit version

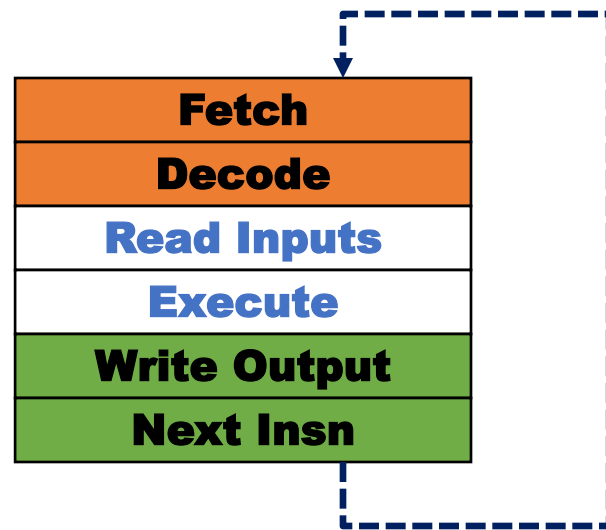
~100-300+ cycles to access Main Memory
Registers & Caches:
Register (CPU Scratchpad): 1 cycle.
L1 Cache: 2-4 cycles, 32kB Inst/Data
L2 Cache: 8-16 cycles, 256kB (combined)
L3 Cache: 20-50 cycles, few MBs



INSTRUCTION EXECUTION MODEL



Fetch → Decode → Execute “cycle”

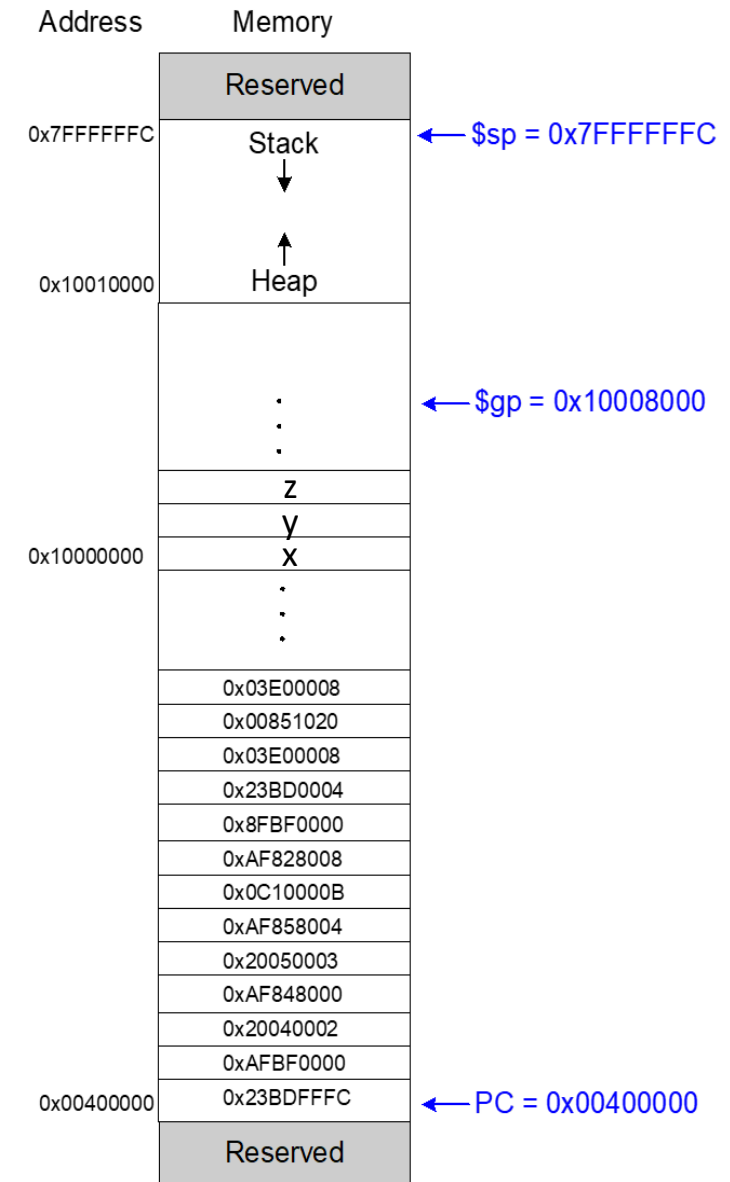
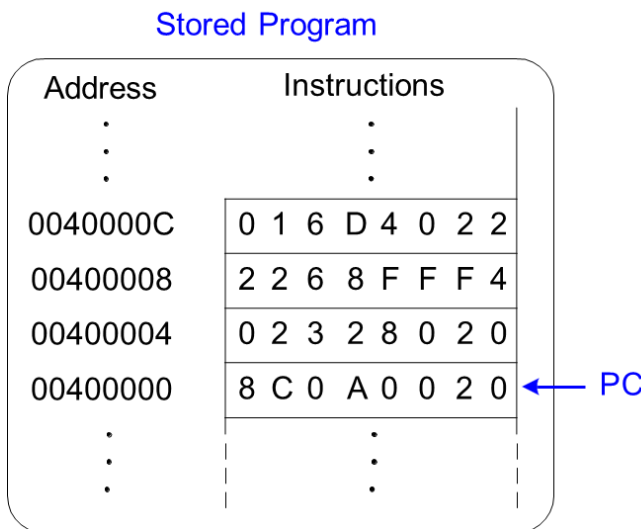


- The computer is just finite state machine
 - **Registers** (few of them, but fast)
 - **Memory** (lots of memory, but slower)
 - **Program counter** (next instruction to execute)
- A computer executes **instructions**
 - **Fetches** next instruction from memory
 - **Decodes** it (figure out what it does)
 - **Reads its inputs** (registers & memory)
 - **Executes** it (adds, multiply, etc.)
 - **Write its outputs** (registers & memory)
 - **Next insn** (adjust the program counter)
- **Program is just “data in memory”**
 - Makes computers programmable (“universal”)

PROGRAM EXECUTION MODEL

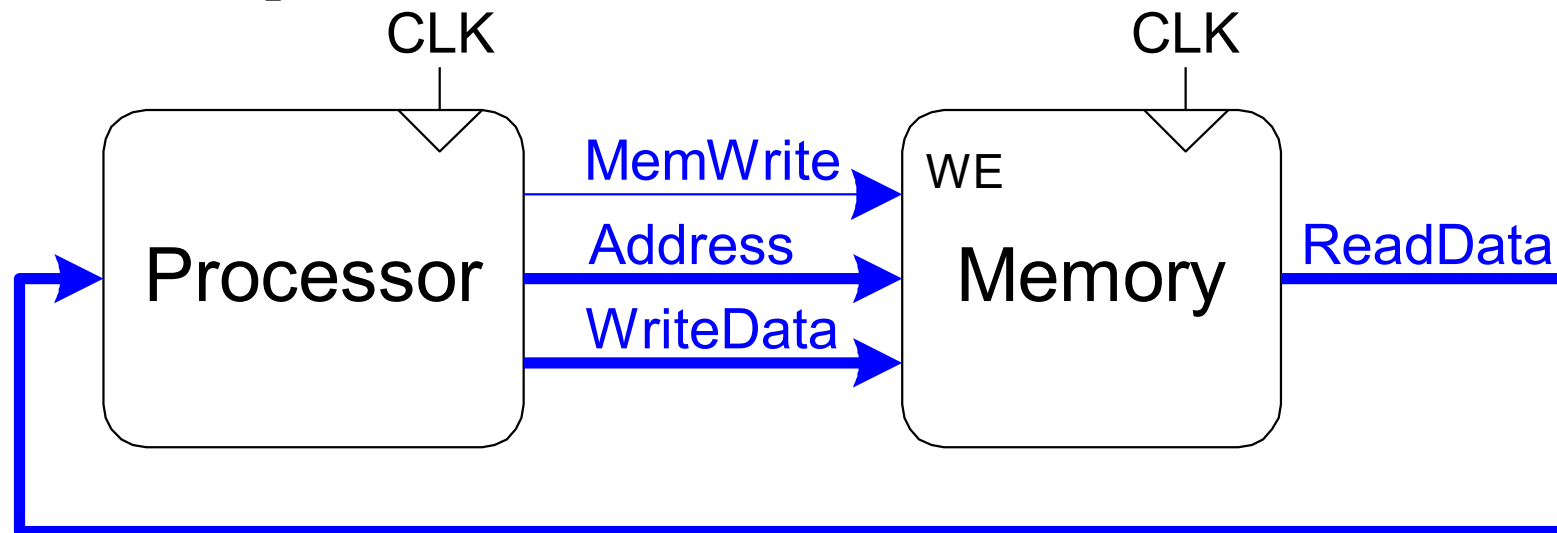
- The processor hardware executes the program:
 - fetches (reads) the instructions from memory in sequence
 - performs the specified operation
 - continues with the next instruction
- The program counter (PC) tracks the current instruction
 - In MIPS, programs typically start at memory address 0x00400000

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022



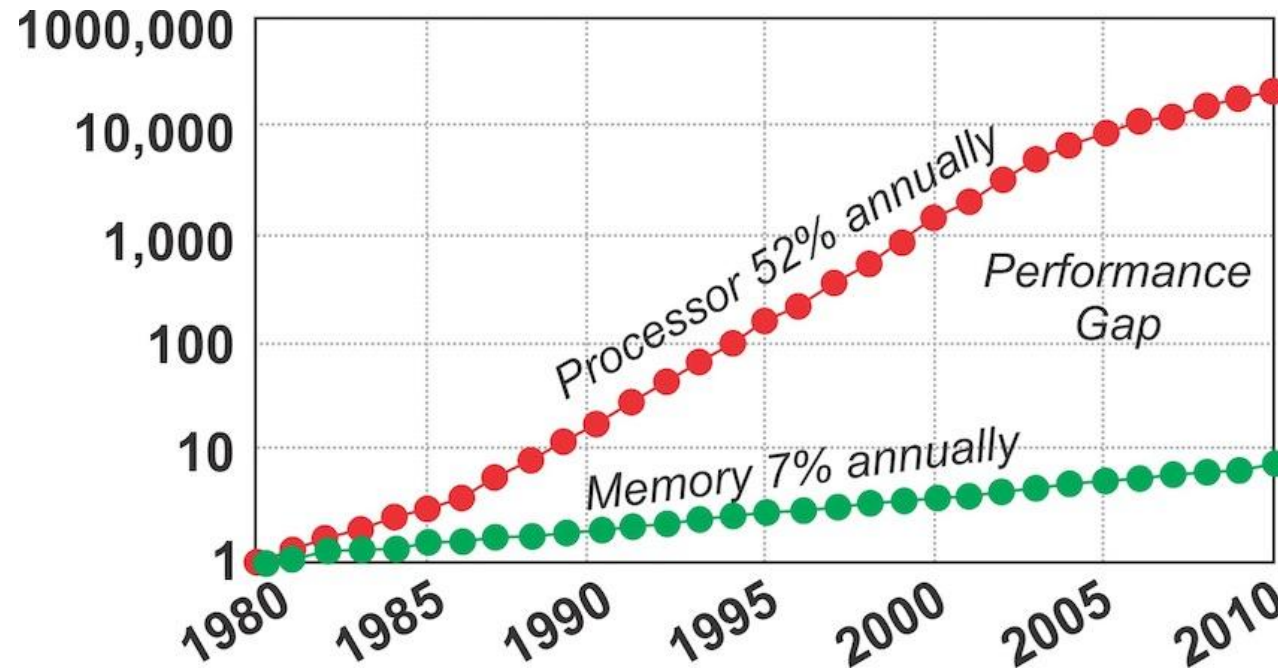
MEMORY SYSTEM -- INTRODUCTION

- Computer performance depends on:
 - Processor performance
 - Memory system performance
- Processor can compute only as fast as memory
 - A 3Ghz processor can execute an “add” operation in 0.33ns
 - Today's “Main memory” latency is more than 33ns (more than 100x slower)
 - Naïve implementation: loads/stores can be 100x slower than other operations



THE MEMORY WALL

We assumed access memory in 1 clock cycle – but hasn't been true since the 1980's



Processors get faster more quickly than memory (note the log scale)

- Processor speed improvement: 35% to 55%
- Memory latency improvement: 7%

Key Question is: How do we bridge this Gap?

- System Design and Optimizations
- Latency Hiding Techniques..

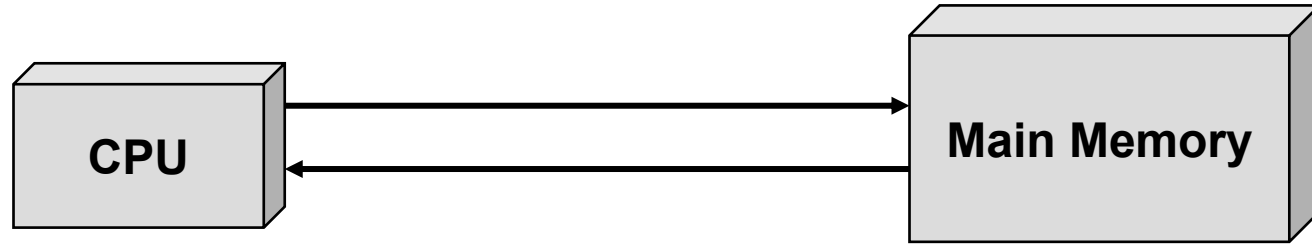
PRINCIPLE OF LOCALITY — TEMPORAL AND SPATIAL

Exploit locality to make memory accesses fast

- **Temporal Locality:** Locality in time
 - If data used recently, likely to use it again soon
 - **How to exploit:** keep recently accessed data as close as possible to the CPU.
- **Spatial Locality:** Locality in space
 - If data used recently, likely to use nearby data soon
 - **How to exploit:** when access data, bring also the nearby data closer to CPU.
- Which memory accesses demonstrate spatial locality?
- Which memory accesses demonstrate temporal locality?

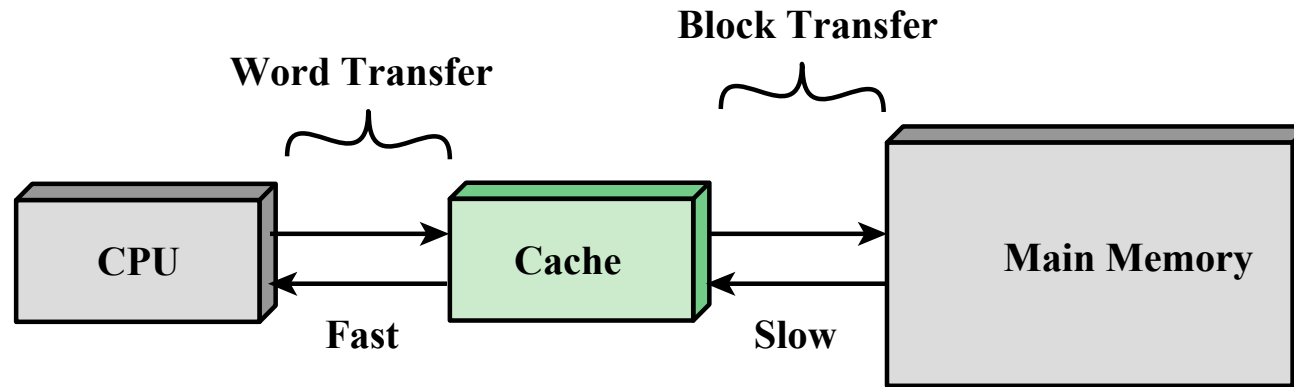
```
#define A_SIZE (1024*16)
int sum = 0;
int arr[A_SIZE];
int c = 0;
for(; c < A_SIZE; ++c) {
    sum += arr[c];
    arr[c] = 0;
}
```


EVOLUTION OF CACHE HIERARCHIES

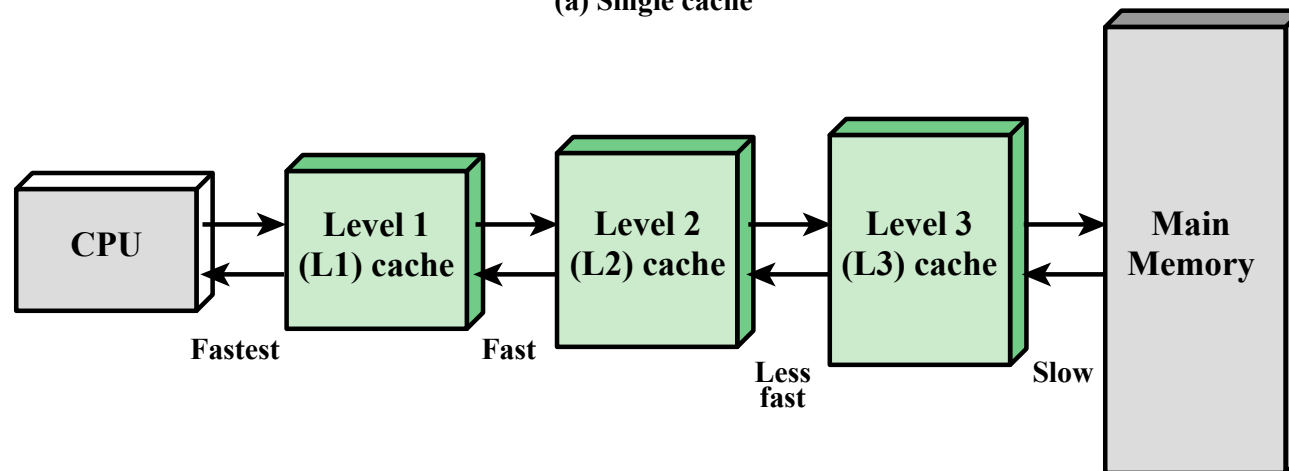


```
#define A_SIZE (1024*16)
int sum = 0;
int X[A_SIZE];
```

```
for(int c = 0; c < A_SIZE; ++c){
    sum += X[c];
    X[c] = 0;
}
```

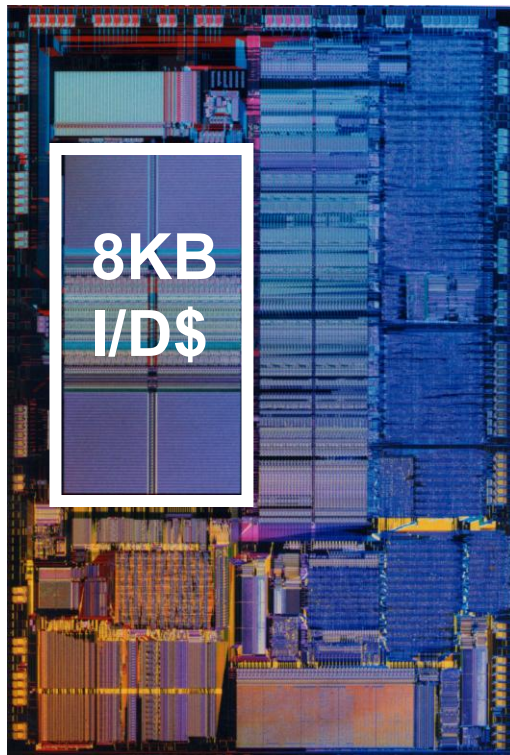


(a) Single cache

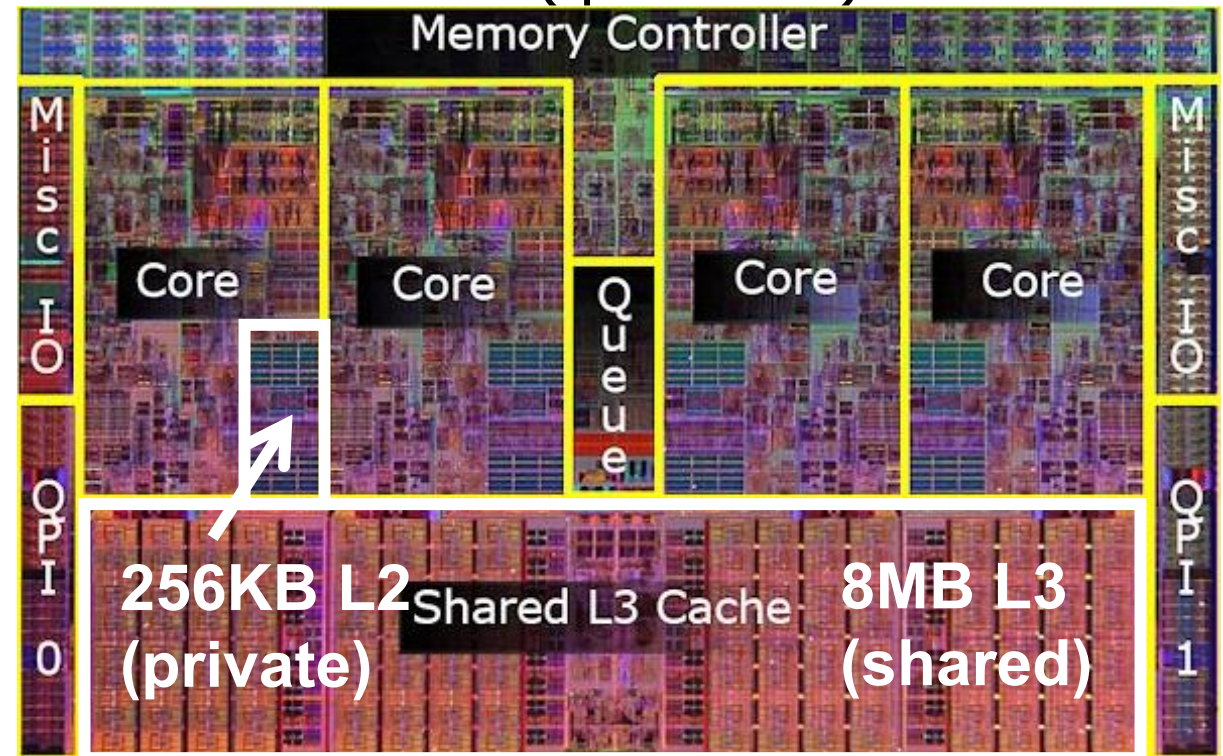


EVOLUTION OF CACHE HIERARCHIES

Intel 486



Intel Core i7 (quad core)



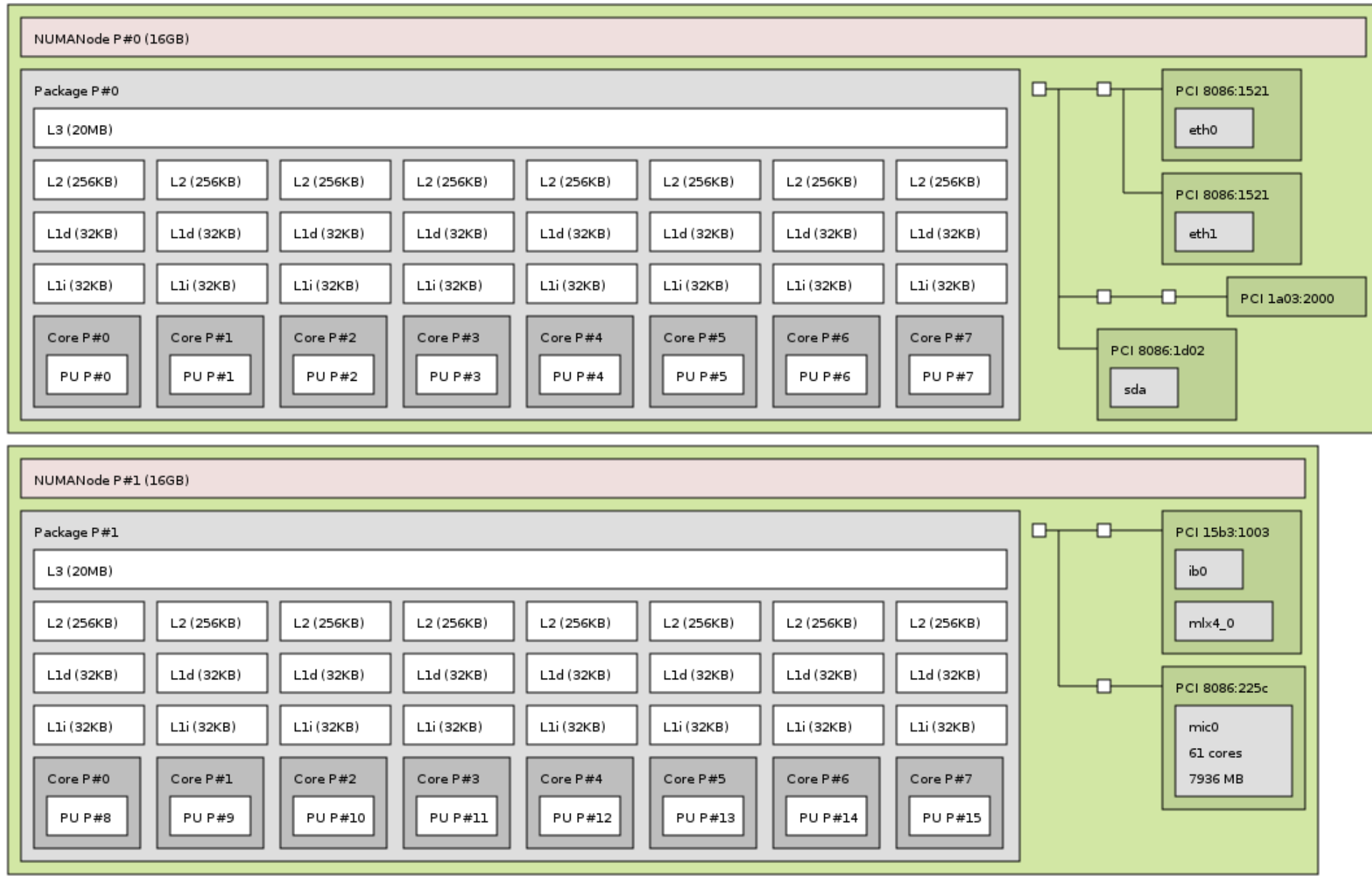
- The 8KB internal cache made i486 twice as fast as the 386 at the same clock rate.

In today's processors:

- Cache accounts to 50–80% of die area and over 80% of transistor count.

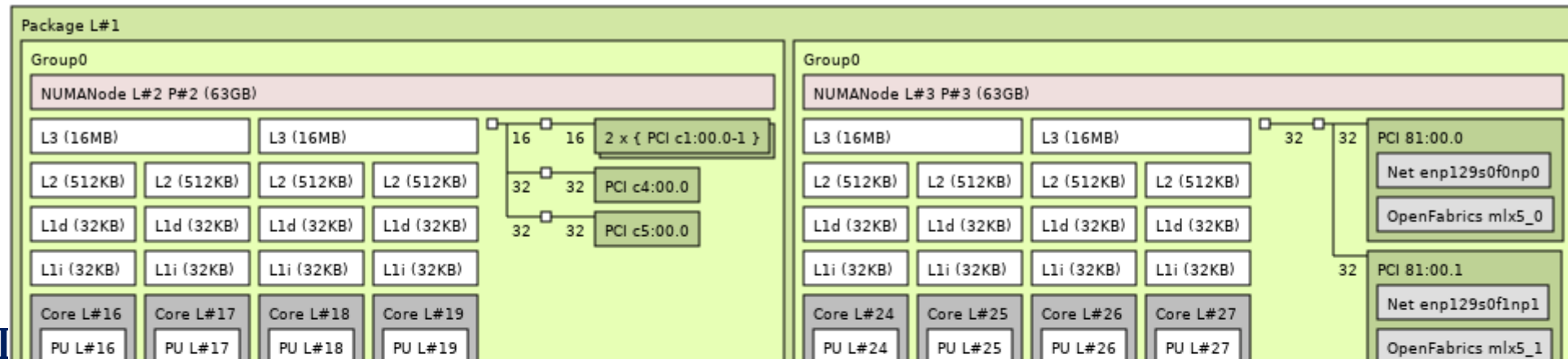
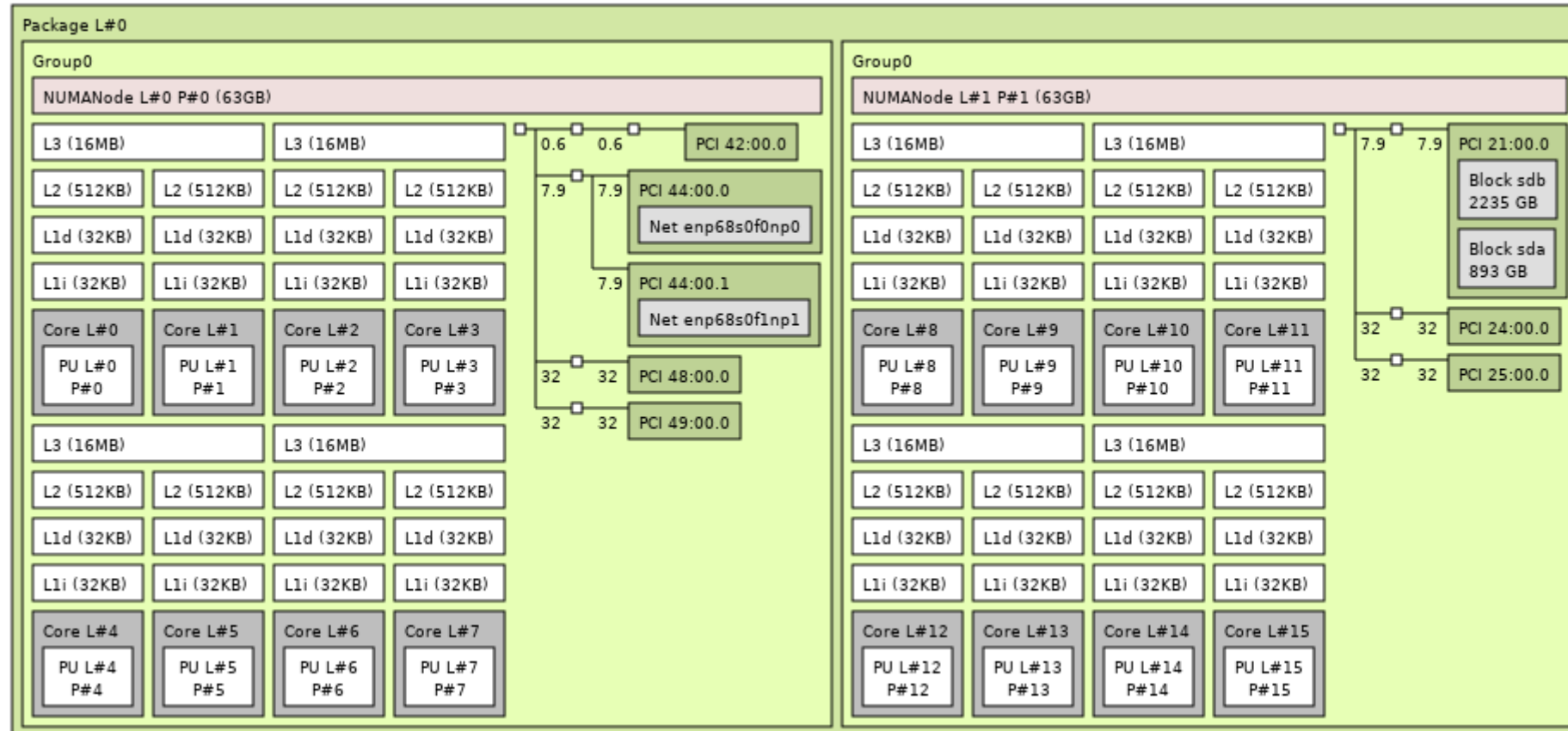
MEMORY HIERARCHY IN MODERN SERVER MACHINES – INTEL XEON (2017)

Machine (32GB total)

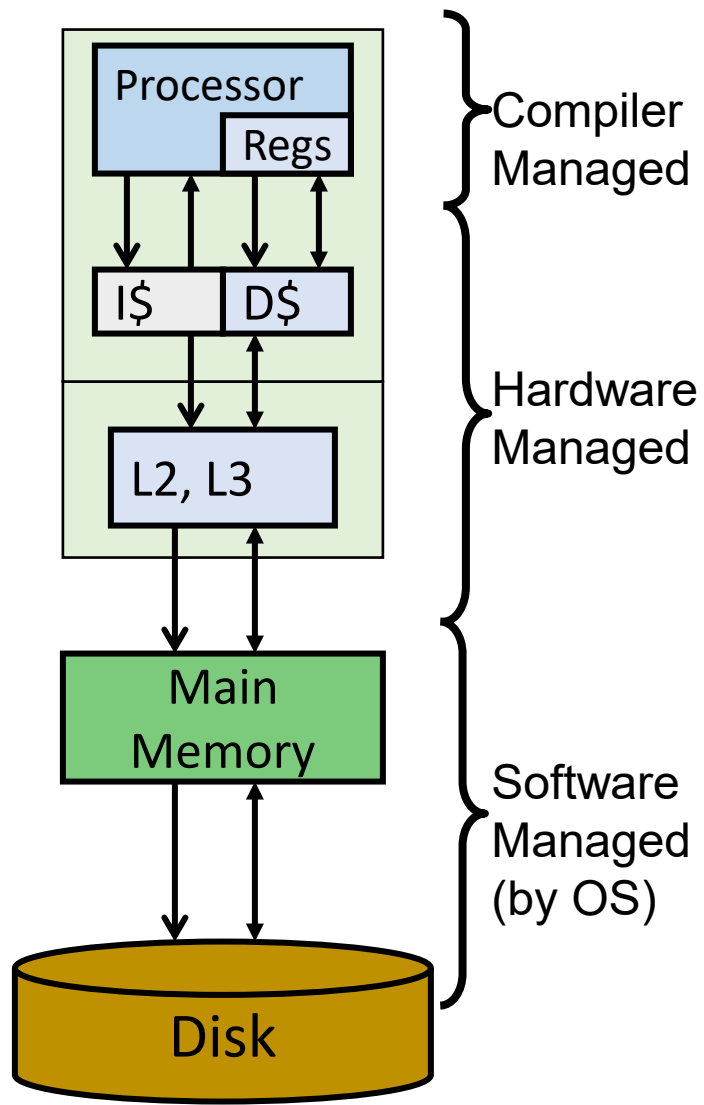


MEMORY HIERARCHY IN MODERN SERVER MACHINES (AMD MILAN 2024)

Machine (252GB total)

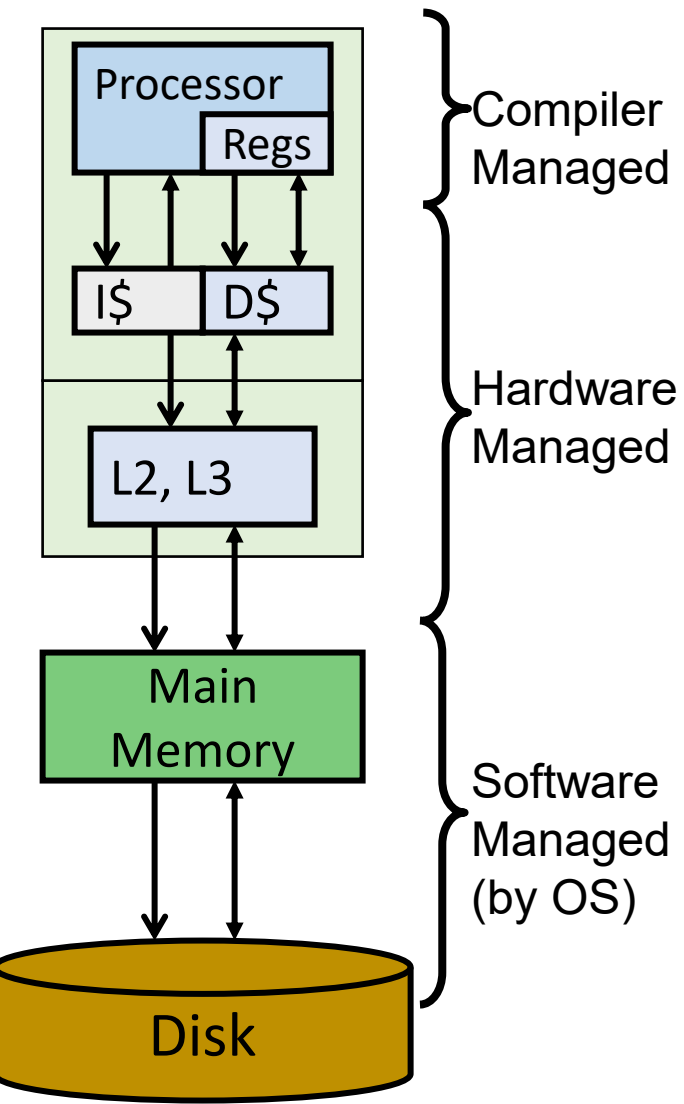


CONCRETE MEMORY HIERARCHY

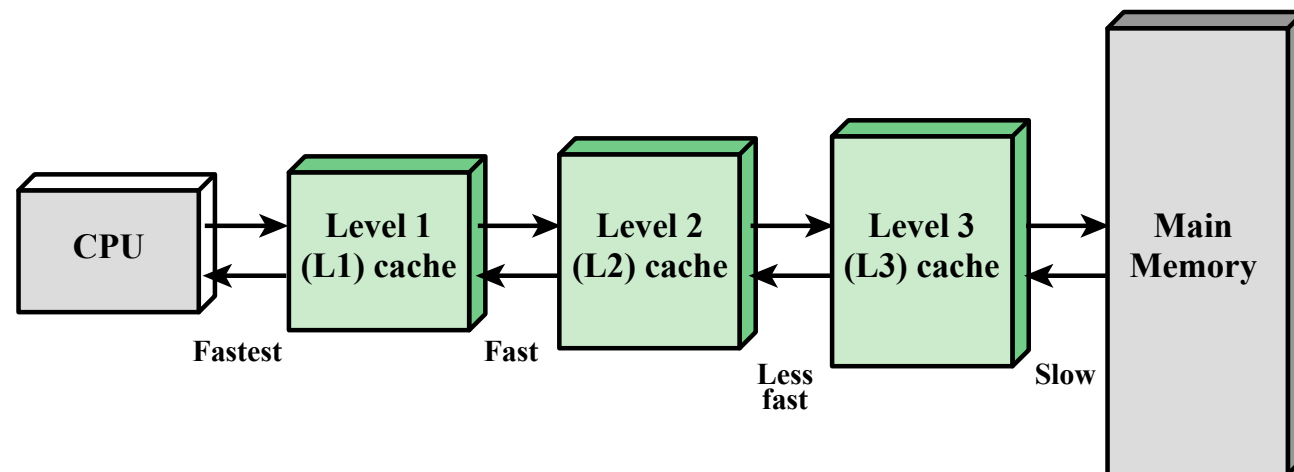


- 0th level: **Registers**
- 1st level: **Primary caches**
 - Split instruction (I\$) and data (D\$)
 - Typically 8KB to 64KB each
- 2nd level: **2nd and 3rd cache** (L2, L3)
 - On-chip, typically made of SRAM
 - 2nd level typically ~256KB to 512KB
 - “Last level cache” typically 4MB to 16MB
- 3rd level: **main memory**
 - Made of DRAM (“Dynamic” RAM)
 - Typically 1GB to 4GB for desktops/laptops
 - Servers can have 100s of GB
- 4th level: **disk (swap and files)**
 - Uses magnetic disks or flash drives

GENERAL MEMORY HIERARCHY



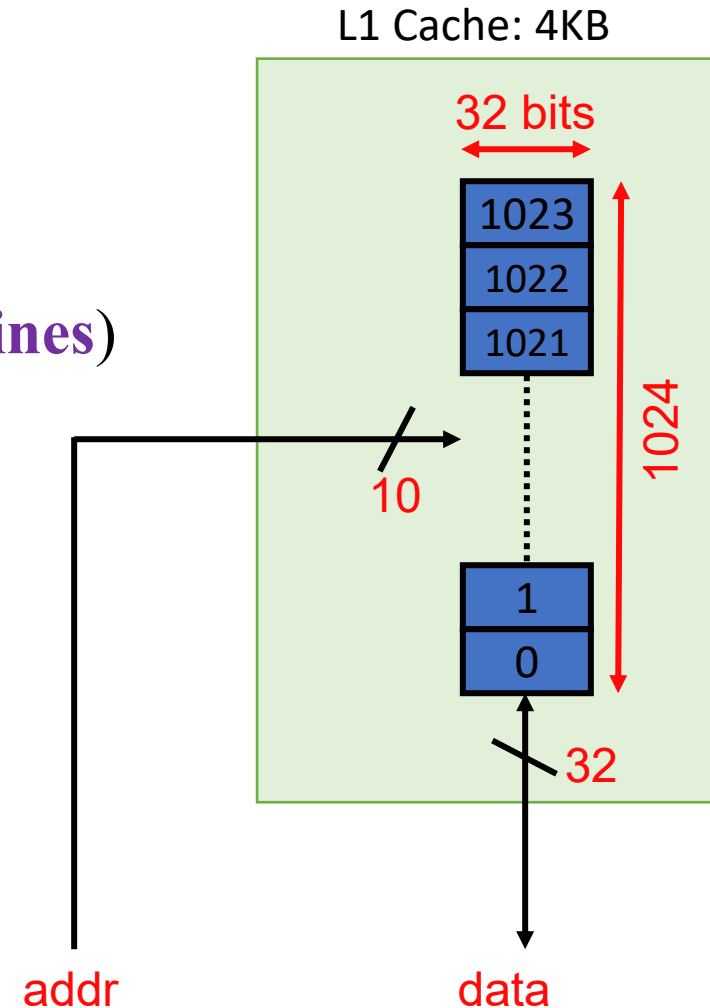
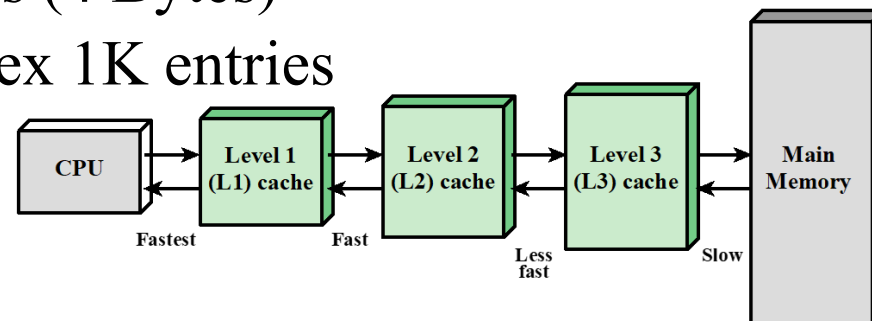
	Speed	Size/Capacity	Access Time	Technology	Price/GB
Register	Fastest	Smallest (~1KB)	<1ns	FFs	--
Cache (L1/L2/L3)	Fast	Small (32KB~MB)	~1-10ns	SRAM	\$100- \$1,000
Main Memory	Slow	Large (1-256 GB)	10-100 ns	DRAM	\$1-\$10
Virtual/Secondary Memory	Slowest	Largest (Few TBs)	0.1-10 ms	SSD/HDD	\$1/\$0.1



Cache Organization

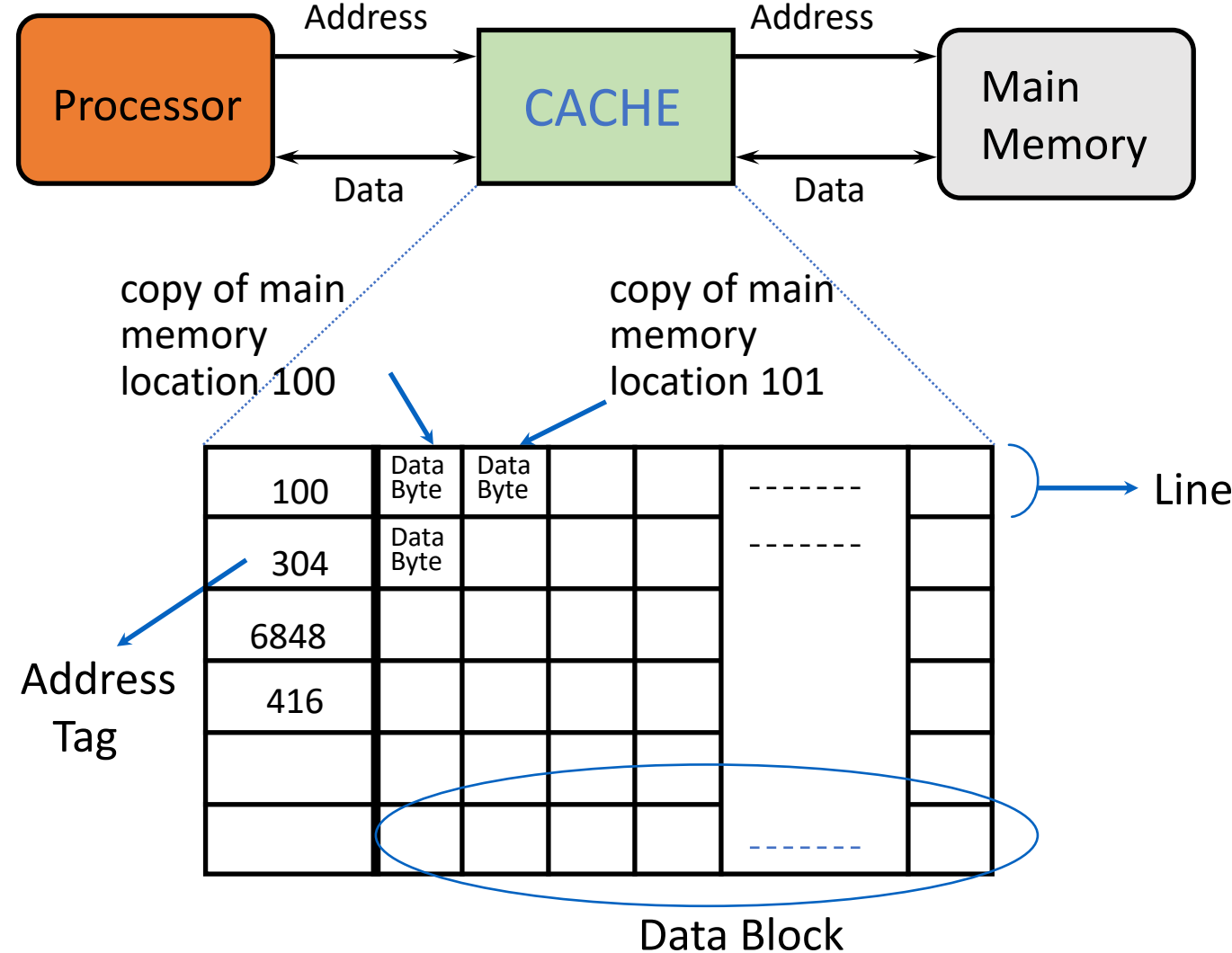
CACHE ORGANIZATION

- Cache – a generic term for any structure that “*memoizes*”.
- Memory Cache is a hardware hashtable – an array of **sets**;
 - Ideally supplies most data to processor
 - Usually holds most recently accessed data
- Logical cache organization: **Sets** and **Blocks**
 - 4KB, organized as 1K (1024 **entries**) of 4B **blocks** (aka **cache lines**)
 - Each block can hold a 4-byte word.
- Physical cache implementation
 - Automatically (Hardware) managed memory, typically SRAM
 - 1K (1024 entries) by 32bits (4 Bytes)
 - 10-bit address input to index 1K entries
 - 32-bit data input/output

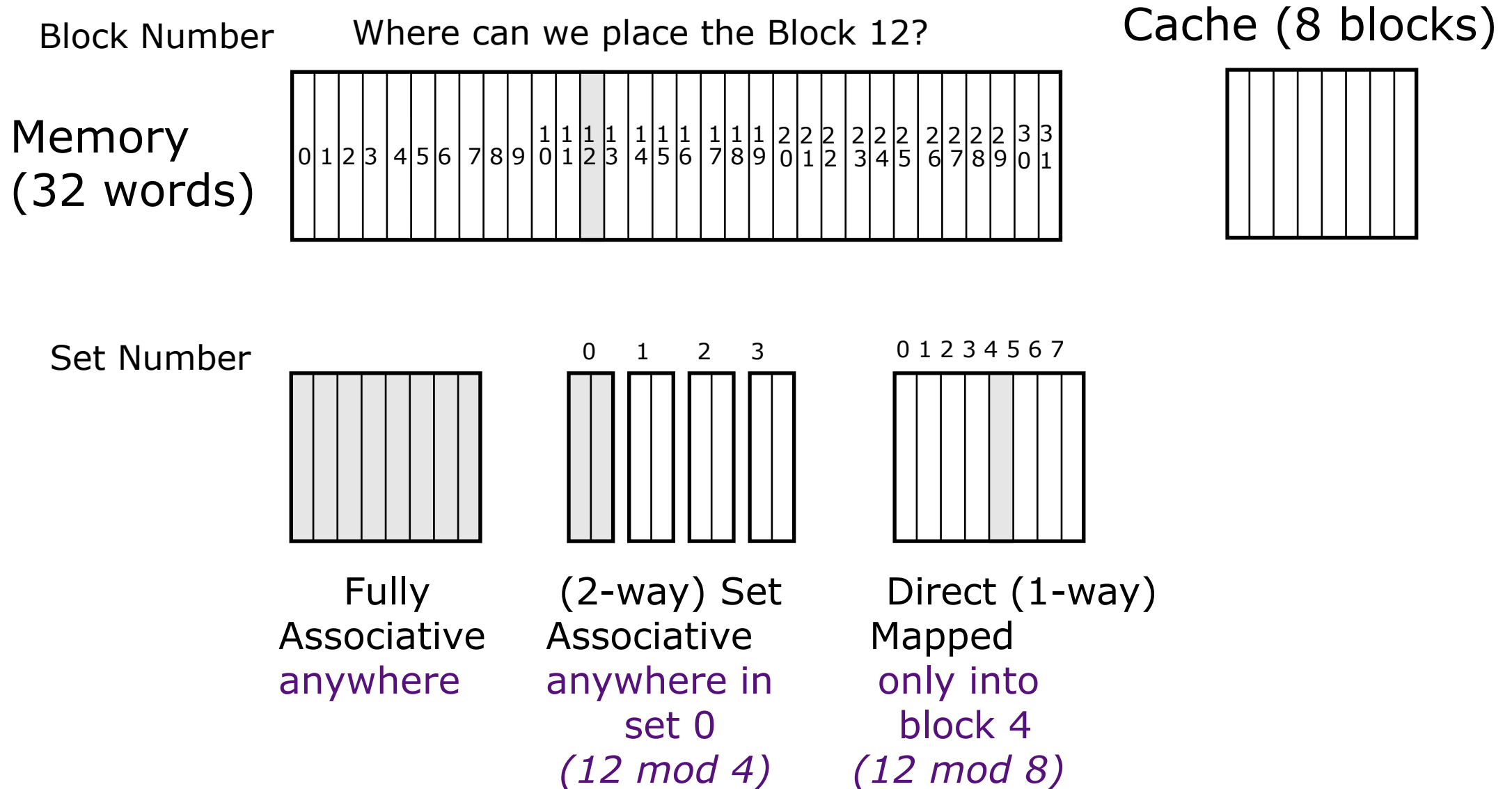


CACHE CHARACTERISTICS AND BASIC CACHE TERMINOLOGY

- **Capacity (C):**
 - Total number of **data bytes** in cache
- **Block size (b):**
 - bytes of data brought into cache at once
- **Number of blocks ($B = C/b$):**
 - number of blocks in cache: $B = C/b$
- **Degree of associativity (N):**
 - number of blocks in a set
- **Number of sets ($S = B/N$):**
 - each memory address maps to exactly one cache set.



DIFFERENT CACHE ORGANIZATIONS – AN EXAMPLE

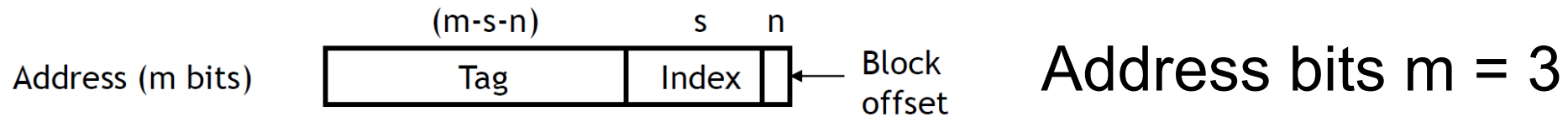


3-BIT ADDRESS, 8B CACHE, 2B BLOCKS

Main memory (8 Bytes, Byte addressable) $\rightarrow$ Total Address bits = $\text{Log}_2(8) = 3$

How do we access cache from the Memory Address bits ?

000	A
001	B
010	C
011	D
100	E
101	F
110	G
111	H



Cache (8 Bytes, Direct Mapped, 2B blocks)

	0	1
00	A	B
01	C	D
10	E	F
11	G	H

2B blocks $\rightarrow b$ (block size) = 2

Block offset bits $n = \text{Log}_2(2) = 1$; $n = 1$ (LSB)

Direct Mapped $\rightarrow$ (N=1 way associative)

#sets = #blocks = $C/b = 4 \rightarrow$ Index bits = 2

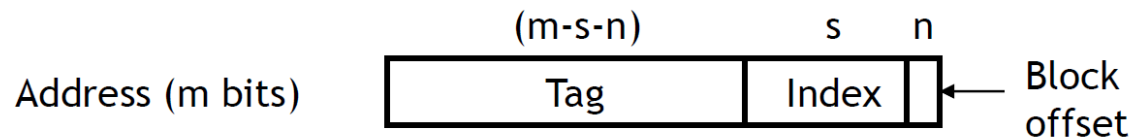
#Tag bits = $(m - s - n) = (3 - 2 - 1) = 0$ (MSB)

4-BIT ADDRESS, 8B CACHE, 2B BLOCKS

Main memory (16 Bytes, Byte addressable) $\rightarrow$ Total Address bits = $\text{Log}_2(16) = 4$

0000	A
0001	B
0010	C
0011	D
0100	E
0101	F
0110	G
0111	H
1000	I
1001	J
1010	K
1011	L
1100	M
1101	N
1110	P
1111	Q

How do we access cache from the Memory Address bits ?



Address bits $m = 4$

Cache (8 Bytes, Direct Mapped, 2B blocks)

		0	1
0	00	A	B
0	01	C	D
0	10	E	F
0	11	G	H

2B blocks $\rightarrow b = 2$

Block offset bits $n = \text{Log}_2(2)=1$; $n=1$ (LSB)

Direct Mapped $\rightarrow$ (N=1 way associative)

#sets = #blocks = $C/b = 4 \rightarrow$ Index bits = 2

#Tag bits = $(m - s - n) = (4-2-1) = 1$ (MSB)

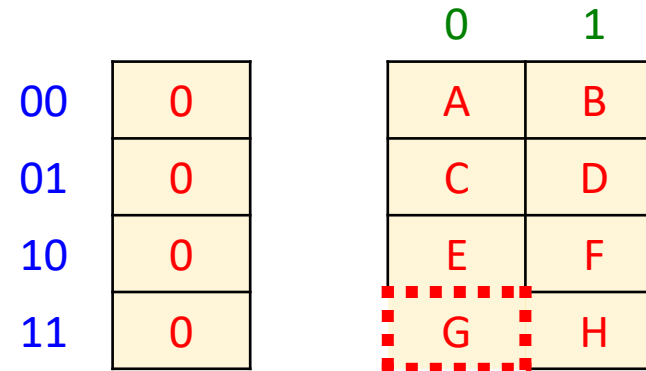
4-BIT ADDRESS, 8B CACHE, 2B BLOCKS

Main memory

0000	A
0001	B
0010	C
0011	D
0100	E
0101	F
0110	G
0111	H
1000	I
1001	J
1010	K
1011	L
1100	M
1101	N
1110	P
1111	Q



Load: 1110 Miss



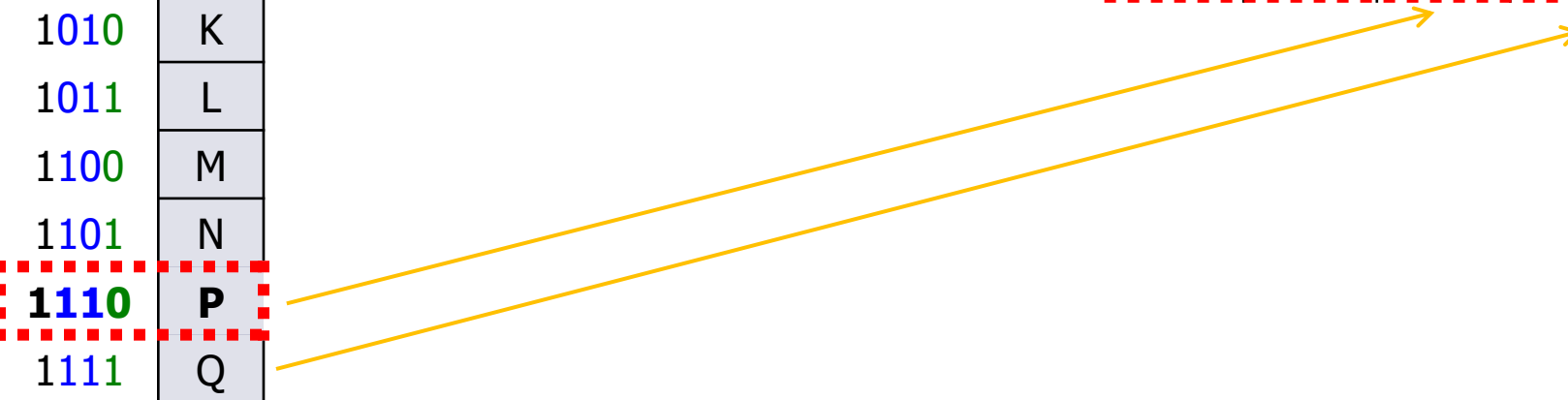
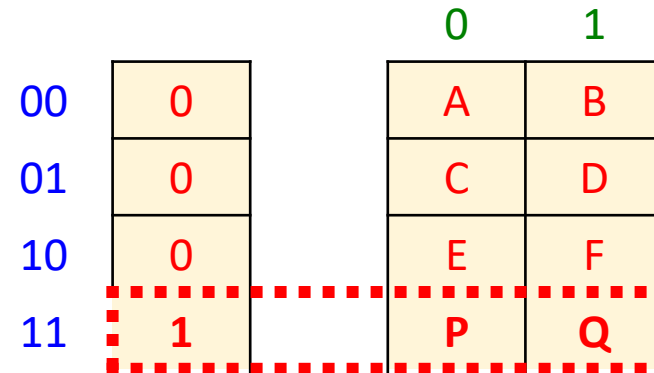
4-BIT ADDRESS, 8B CACHE, 2B BLOCKS

Main memory

0000	A
0001	B
0010	C
0011	D
0100	E
0101	F
0110	G
0111	H
1000	I
1001	J
1010	K
1011	L
1100	M
1101	N
1110	P
1111	Q



Load: 1110 Miss



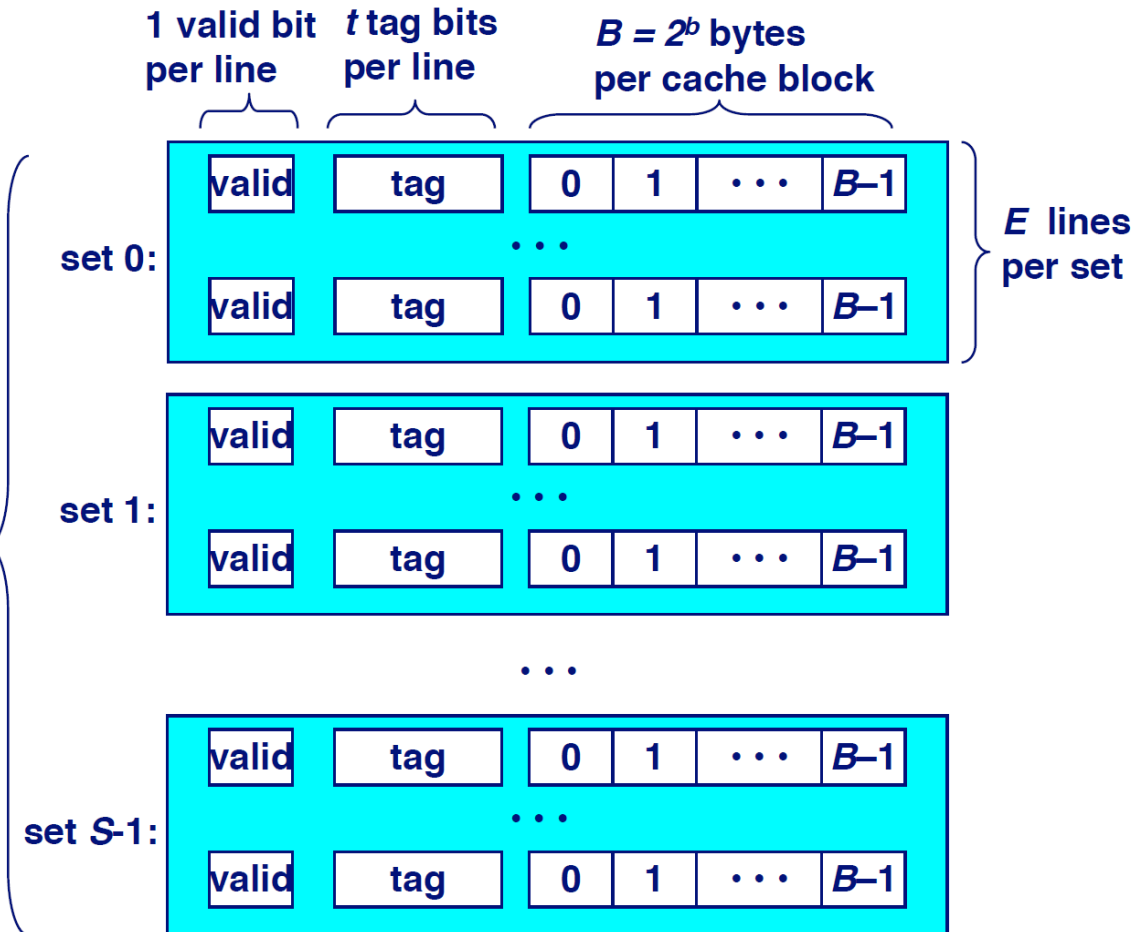
CACHE DESIGN

Cache is an array of sets.

Each set contains one or more lines.

Each line holds a block of data.

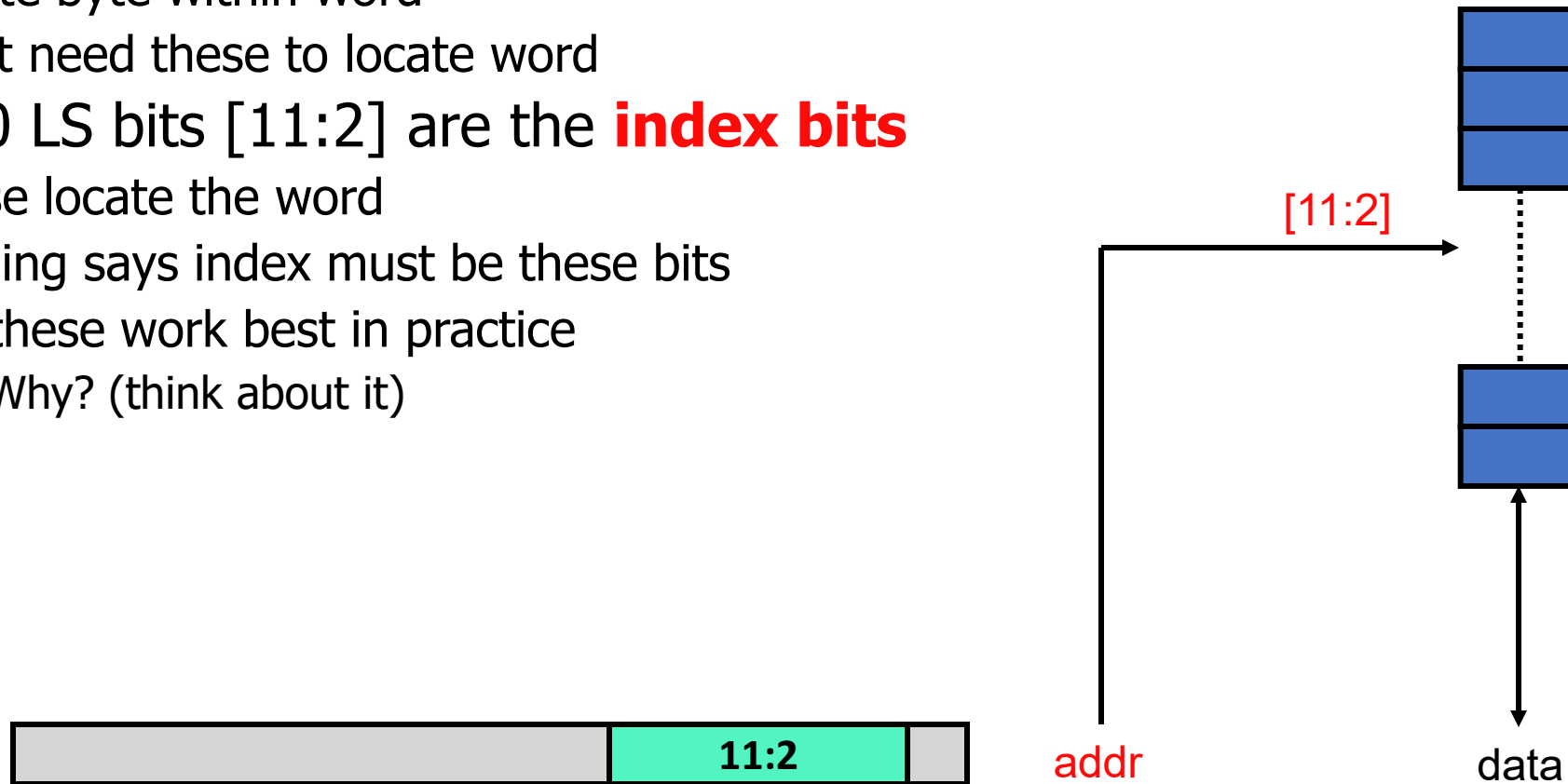
$S = 2^s$ sets



Cache size: $C = B \times E \times S$ data bytes

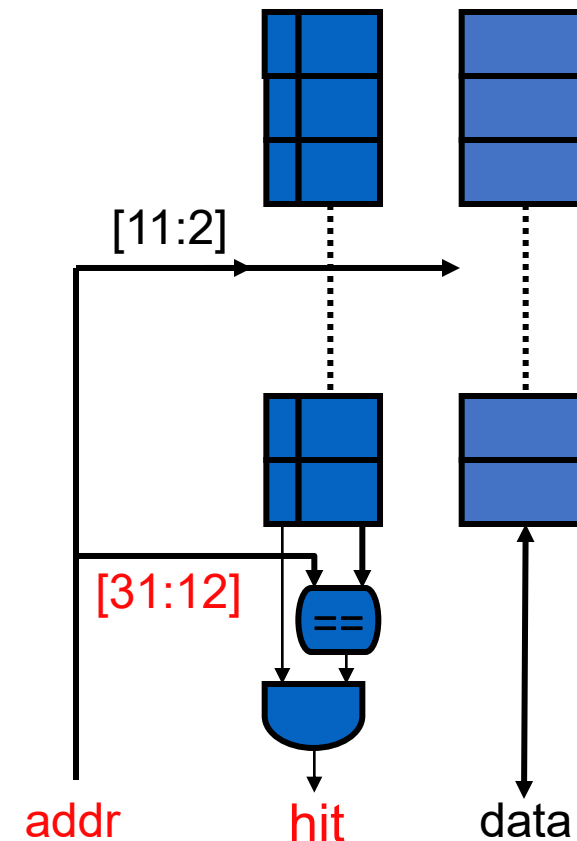
LOOKING UP A BLOCK

- Q: which 10 of the 32 address bits to use?
- A: bits [11:2]
 - 2 least significant (LS) bits [1:0] are the **offset bits**
 - Locate byte within word
 - Don't need these to locate word
 - Next 10 LS bits [11:2] are the **index bits**
 - These locate the word
 - Nothing says index must be these bits
 - But these work best in practice
 - Why? (think about it)



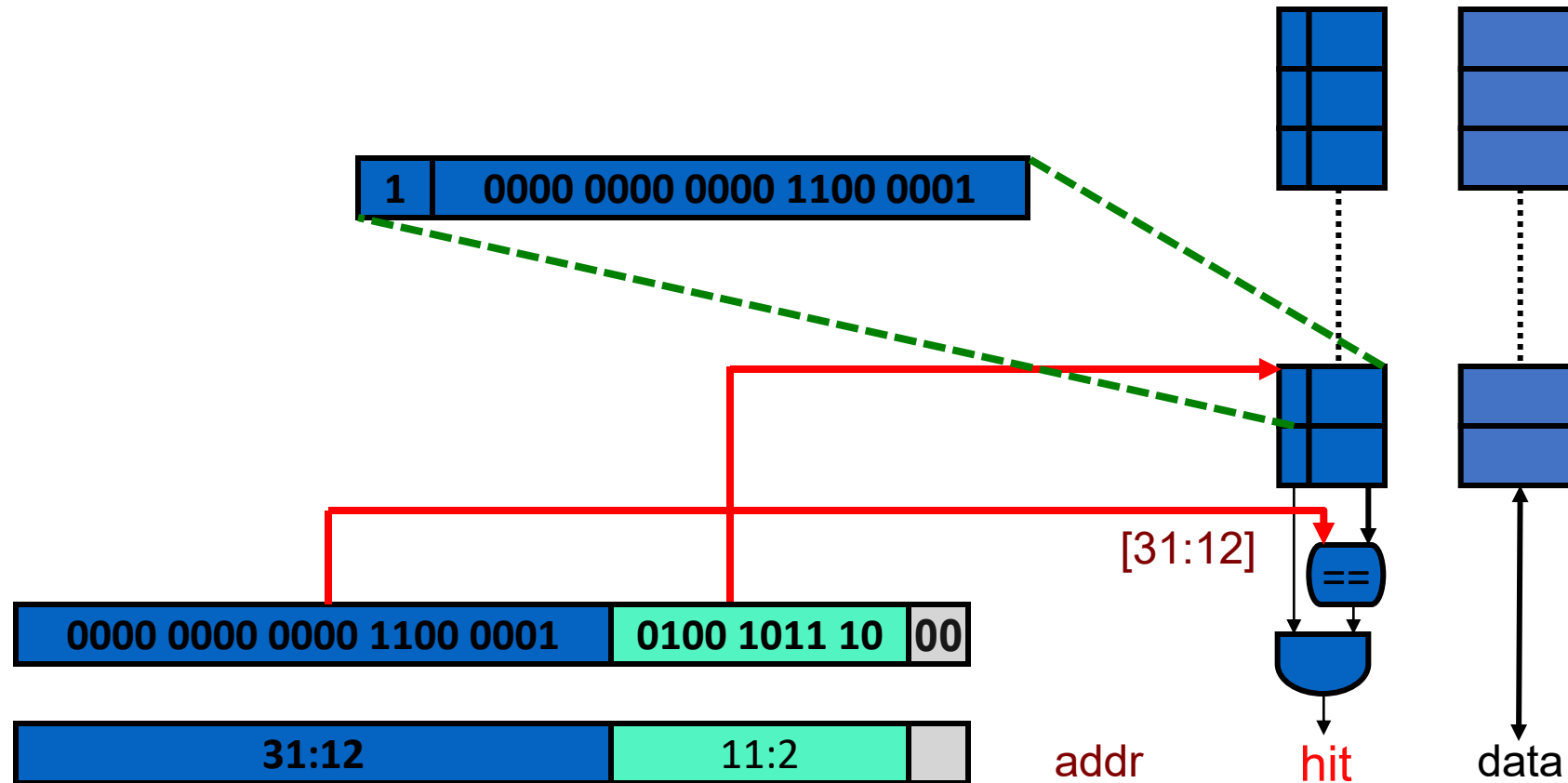
IS THIS THE BLOCK YOU'RE LOOKING FOR?

- Each cache row corresponds to 2^{20} blocks
 - How to know which if any is currently there?
 - Tag each cache word with remaining address bits [31:12]
- Build separate and parallel **tag array**
 - 1K by 21-bit SRAM
 - 20-bit (next slide) **tag** + 1 **valid bit**
- Lookup algorithm
 - Read tag indicated by index bits
 - If tag matches & valid bit set:
 - then: Hit → data is good
 - else: Miss → data is garbage, wait...



A CONCRETE EXAMPLE

- Lookup address x000C14B8
 - Index = $\text{addr}[11:2] = (\text{addr} \gg 2) \& \text{x3FF} = \text{x12E}$
 - Tag = $\text{addr}[31:12] = (\text{addr} \gg 12) = \text{x000C1}$



DIRECT MAPPING VS N-WAY SET ASSOCIATIVE CACHE

- **Capacity (C):**

- number of data bytes in cache

- **Block size (b):**

- bytes of data brought into cache at once

- **Number of blocks ($B = C/b$):**

- number of blocks in cache: $B = C/b$

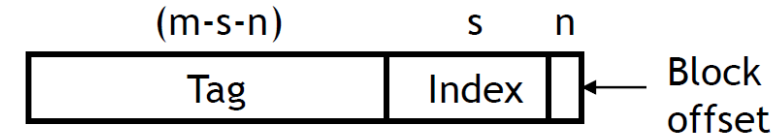
- **Degree of associativity (N):**

- number of blocks in a set

- **Number of sets ($S = B/N$):**

- each memory address maps to exactly one cache set

Address (m bits)



$$C = B \times b$$

$$C = \{S \times N\} \times b$$

$$\text{set index} = (\text{block address}) \bmod (S)$$

CONSIDER MEMORY: 32-BIT ADDRESS AND CACHE 32KB, 4B BLOCKS

Cache organization	Degree of Associativity (N)	Num of sets (S=B/N)	#bits for block offset	#bits for set index	# bits for Tag	Tag Overhead (Tag bits/Block bits)
Direct-Mapped	1	8K	2	13	17	17/32
2-way Associative	2	4K	2	12	18	18/32
4-way Associative	4	2K	2	11	19	19/32
Fully Associative	8	1	2	0	30	30/32

BLOCK SIZE AND TAG OVERHEAD

- **Note: Tags are not optional and incur overhead**

- 4KB cache with 1024 4B blocks?

- 4B blocks $\rightarrow$ 2-bit offset, 1024 blocks $\rightarrow$ 10-bit index
- 32-bit address $-$ 2-bit offset $-$ 10-bit index = 20-bit tag
- **20-bit tag / 32-bit block = 63% overhead**
- **32-bit block / 52-bit line = 62% efficiency**

- 4KB cache with 512 8B blocks

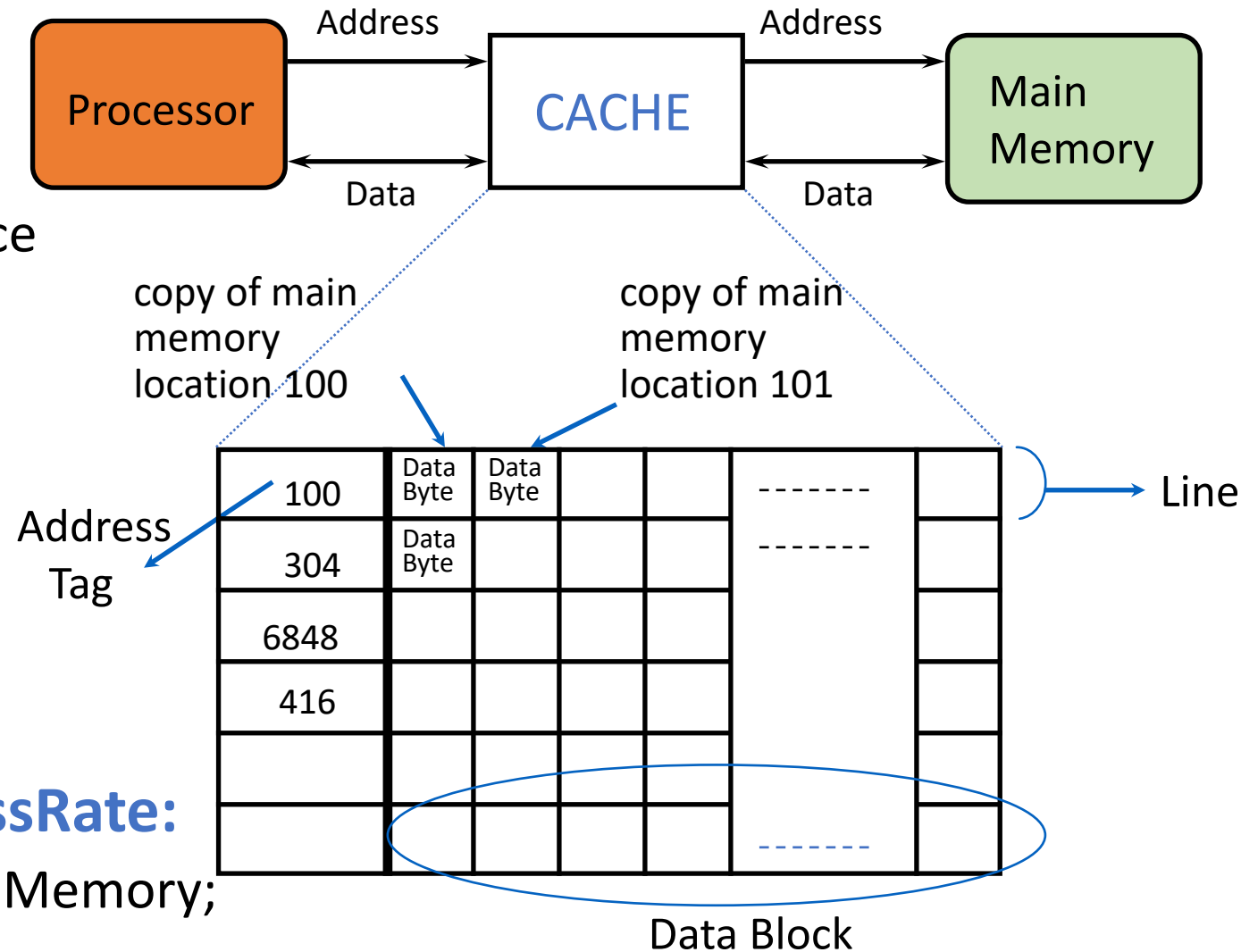
- 8B blocks $\rightarrow$ 3-bit offset, 512 frames $\rightarrow$ 9-bit index
- 32-bit address $-$ 3-bit offset $-$ 9-bit index = 20-bit tag
- **20-bit tag / 64-bit block = 32% overhead**
- **64-bit block / 84-bit line = 76% efficiency**
- Notice: tag size is same, but data size is twice as big

- A realistic example: 64KB cache with 64B blocks

- **16-bit tag / 512-bit block = $\sim$ 3.125% overhead**
- **512-bit block / 528-bit line = 97% efficiency**

TYPICAL MEMORY CACHE AND BASIC CACHE TERMINOLOGY

- **Capacity (C):**
 - number of data bytes in cache
- **Block size (b):**
 - bytes of data brought into cache at once
- **Number of blocks ($B = C/b$):**
 - number of blocks in cache: $B = C/b$
- **Hit vs Miss:**
 - Data in Cache vs not in Cache
- **Hit Time, Hit Rate vs Miss Time, MissRate:**
 - Time to fetch data from cache vs Main Memory;
 - How often data is fetched from cache?



QUESTION (ENDSEM 2023-24)

Arrays *a*, *b* and *c* are stored in row-major order in memory locations starting from 0x1000, 0x1400, 0x1800 respectively. Each of the element of the matrices are four-byte (32-bit) wide. Consider that the processor has a single 16KB direct-mapped data cache with 32-byte wide line size. Before starting the execution of the code fragment above, the cache is all invalid and therefore the very first access shall result in a cache miss. Answer the following questions.

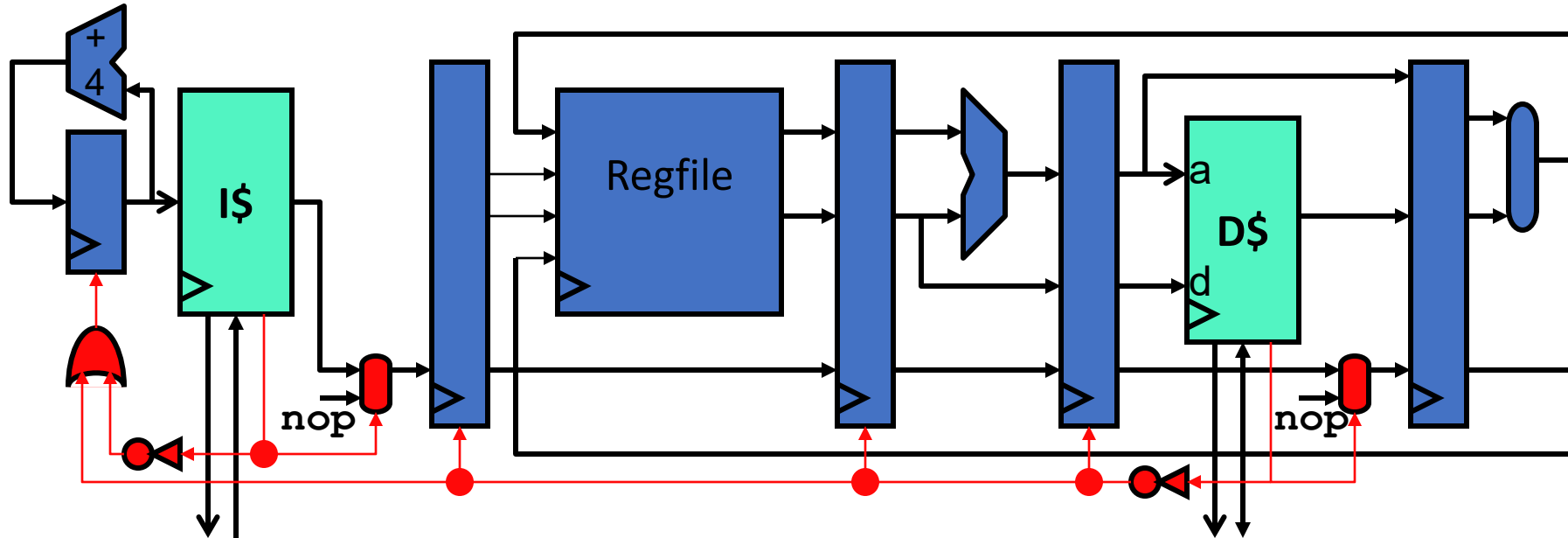
```
#define N 16
for (register int i = 0; i < N; i++){
    for (register int j = 0; j < N; j++) {
        c[i][j] = 0;
        for (register int k = 0; k < N; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
    }
}
```

- How many lines are there in the cache?
- What is the size of tag to be stored along with each cache line? Use two bits for cache valid and cache dirty status. Assume processor uses 32-bit memory addresses.

HANDLING A CACHE MISS

- What if requested data isn't in the cache?
 - How does it get in there?
- **Cache controller**: finite state machine
 - Remembers miss address
 - Accesses next level of memory
 - Waits for response
 - Writes data/tag into proper locations
- Bringing a missing block into the cache is a **cache fill**

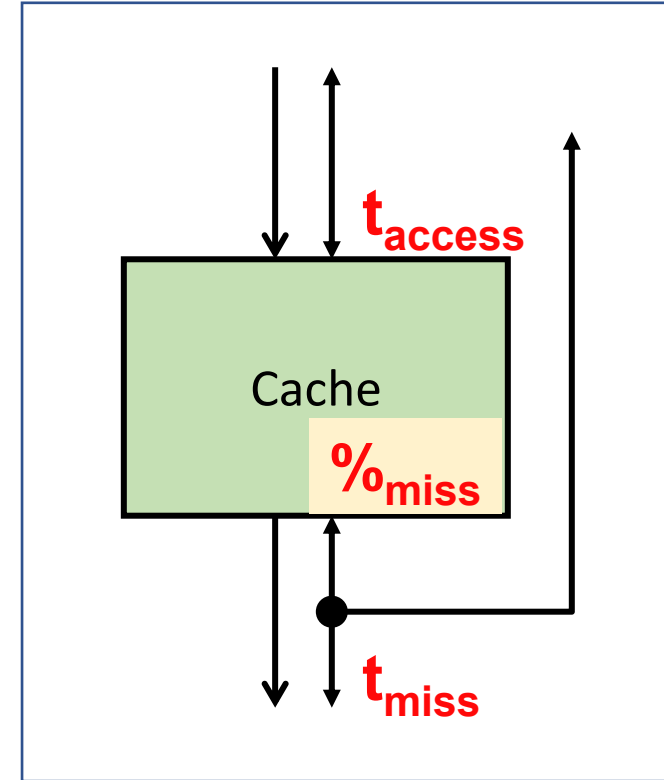
CACHE MISSES AND PIPELINE STALLS



- I\$ and D\$ misses stall pipeline just like data hazards
 - Stall logic driven by miss signal
 - Cache “logically” re-evaluates hit/miss every cycle
 - Block is filled → miss signal de-asserts → pipeline restarts

CACHE PERFORMANCE

- **Access**: read or write to cache
- **Hit**: desired data found in cache
- **Miss**: desired data not found in cache
 - Must get from another component (next in the hierarchy)
 - No notion of *miss* in “register file” and “main_memory”
- **Fill**: action of placing data into cache
- **%_{miss}** (miss-rate): #misses / #accesses
- **t_{access}**: time to check cache. If hit, we're done.
- **t_{miss}**: time to read/fill data into cache
- Performance metric: average memory access time (AMAT)



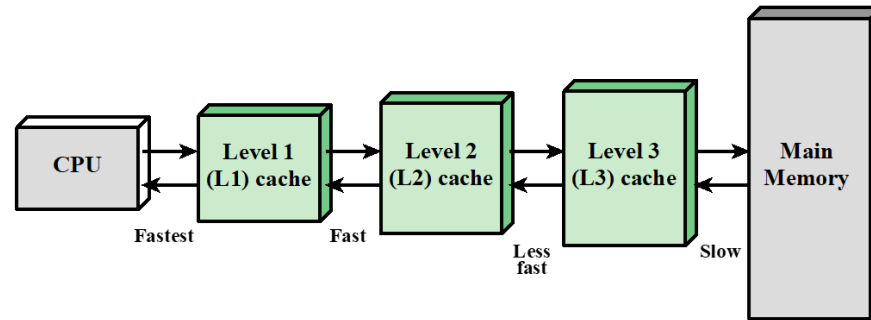
$$t_{\text{avg}} = t_{\text{access}} + (\%_{\text{miss}} * t_{\text{miss}})$$

CACHE -- HIERARCHICAL LATENCY ANALYSIS

- Memory hierarchy (level i) has a technology-intrinsic access time of t_i .
- Except for the *outer-most hierarchy (MM)*, when looking for a given address there is
 - a chance (hit-rate h_i) you “hit” with access time is t_i
 - a chance (miss-rate m_i) you “miss” and fetch from next level with access time $t_i + T_{i+1}$
 - $h_i + m_i = 1$
- Thus

$$T_i = h_i \cdot t_i + m_i \cdot (t_i + T_{i+1})$$

$$T_i = t_i + m_i \cdot T_{i+1}$$



Note: h_i and m_i are the hit-rate and miss-rate of the references that missed at L_{i-1}

- Recursive cache access latency equation.

$$T_i = t_i + m_i \cdot T_{i+1}$$

CPI CALCULATION WITH CACHE MISSES

- Parameters

- Simple pipeline with base CPI of 1
- Instruction mix: 30% loads/stores
- I\$: %_{miss} = 2%, t_{miss} = 10 cycles
- D\$: %_{miss} = 10%, t_{miss} = 10 cycles

$$\text{CPUTime} = \text{IC} \times \text{CPI}_{\text{new}} \times \text{ClockCycleTime}$$

$$\text{CPI}_{\text{new}} = \text{CPU Exec. Cycles} + \text{Memory Stall Cycles (MSC)}$$

$$\text{MSC} = \# \text{MemAccess} \times \text{MissRate} \times \text{MissPenalty}$$

- What is new CPI?

- $\text{CPI}_{\text{I\$}} = \%_{\text{missI\$}} \times t_{\text{miss}} = 0.02 \times 10 \text{ cycles} = 0.2 \text{ cycle}$
- $\text{CPI}_{\text{D\$}} = \%_{\text{load/store}} \times \%_{\text{missD\$}} \times t_{\text{missD\$}} = 0.3 \times 0.1 \times 10 \text{ cycles} = 0.3 \text{ cycle}$
- $\text{CPI}_{\text{new}} = \text{CPI} + \text{CPI}_{\text{I\$}} + \text{CPI}_{\text{D\$}} = 1 + 0.2 + 0.3 = 1.5$

- Note: Two factors impact CPI: **Miss Rate** and **Miss Penalty**

- What is new CPU time?

$$\text{CPUtime} = \text{IC} * (\text{CPI}_{\text{new}}) * \text{Clock Cycle time}$$

Q1: Processor has two level caches L1, L2 and memory. The access time of is 1ns, 10ns, 100ns. Given the hit rate in L1 and L2 cache is 80% and 90% respectively. Compute the average memory access time.

AMAT=?

Q2. If we remove the L1 cache in this configuration, what will be the average memory access time.

QUESTION (ENDSEM 2023-24)

```
#define N 16
for (register int i = 0; i < N; i++){
    for (register int j = 0; j < N; j++) {
        c[i][j] = 0;
        for (register int k = 0; k < N; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
    }
}
```

- If the access to data cache takes one clock cycle and the access to memory takes 30 clock cycles, what will be the total time taken to read and write data while executing this program fragment?
- During the execution of the program, how many data memory access (32-bit wide) shall take place?
- Out of these memory accesses how many shall be for reading and how many shall be for writing of data.
- How many cache misses shall take place in total for access of data?

CACHE MISS AND WAYS TO REDUCE CACHE MISS RATE

- Understanding the types of cache miss: The Three C's + '*C*'
 - **Compulsory (Cold-start)**: the first access to a block in cache
 - **Capacity**: desired data cannot be contained in the cache (size limitation)
 - **Conflict (Collision)**: desired data conflicts with other content in the cache.
 - **Coherence**: miss due to external invalidations (multiprocessor architecture)

Question: How do we minimize these Cache Misses?

Rather, How do we better utilize these caches?

Reduce the Miss Rate and/or Reduce the **Miss Penalty**
*May Also **Reduce the Hit Time***

TECHNIQUES TO REDUCE THE CACHE MISSES

1. *Larger Caches*
2. *Larger Block Size*
3. *Higher associativity*
4. Victim and Pseudo-associative caches
5. Prefetching of Instruction and Data
6. Code optimizations (Developer/Compiler)

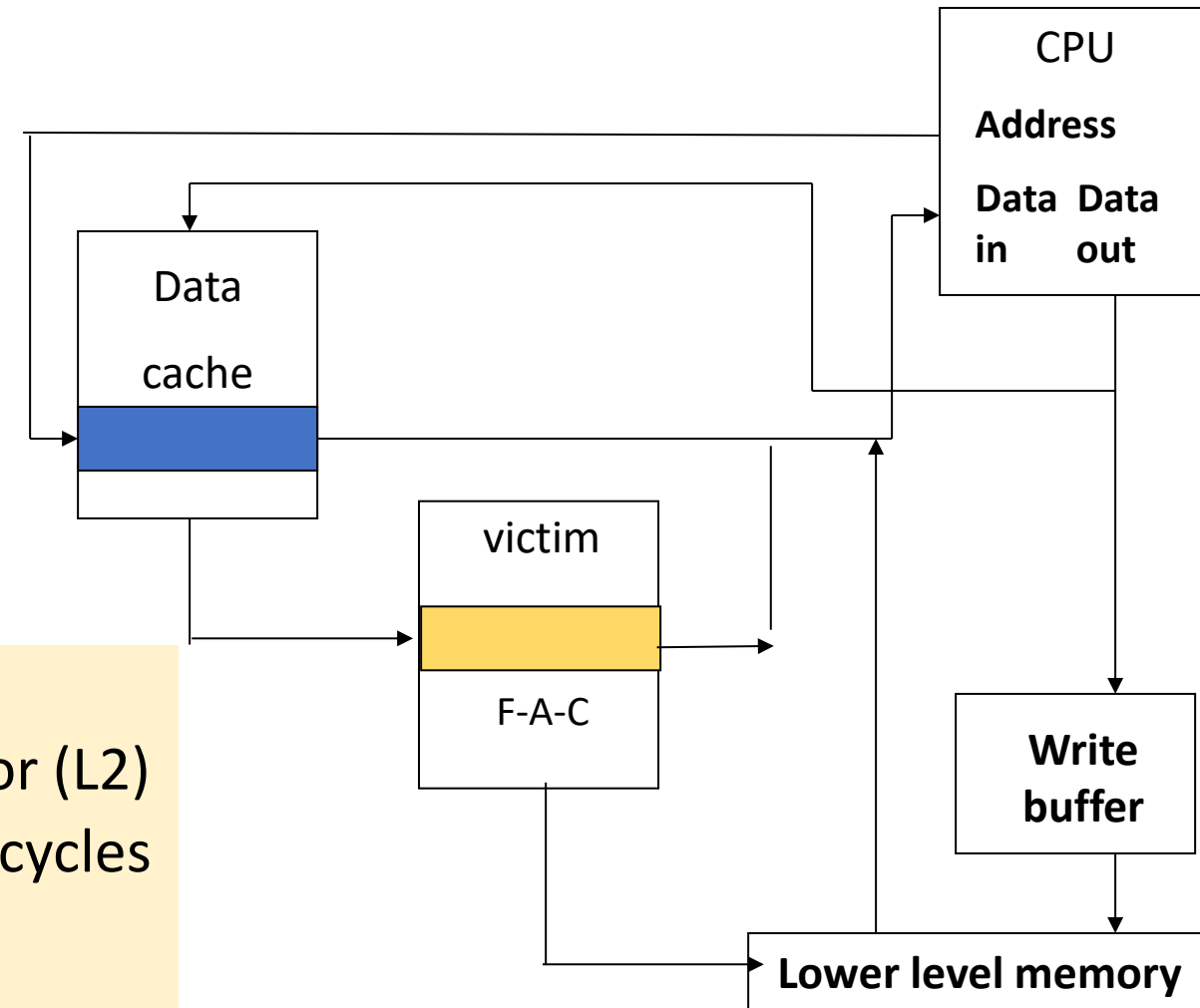
VICTIM CACHES

- Minimize penalty for frequent Conflict/Capacity misses.
- Save data discarded from cache into a additional buffer – victim cache



1. *Simplicity favours regularity*
2. *Make the common case fast*
3. *Smaller is faster*
4. *Good design demands good compromises*

- Used in MIPS R1000 L2 cache
- Better for caches not tied directly to processor (L2)
- CPU pipeline design is hard if hit takes 1 or 2 cycles

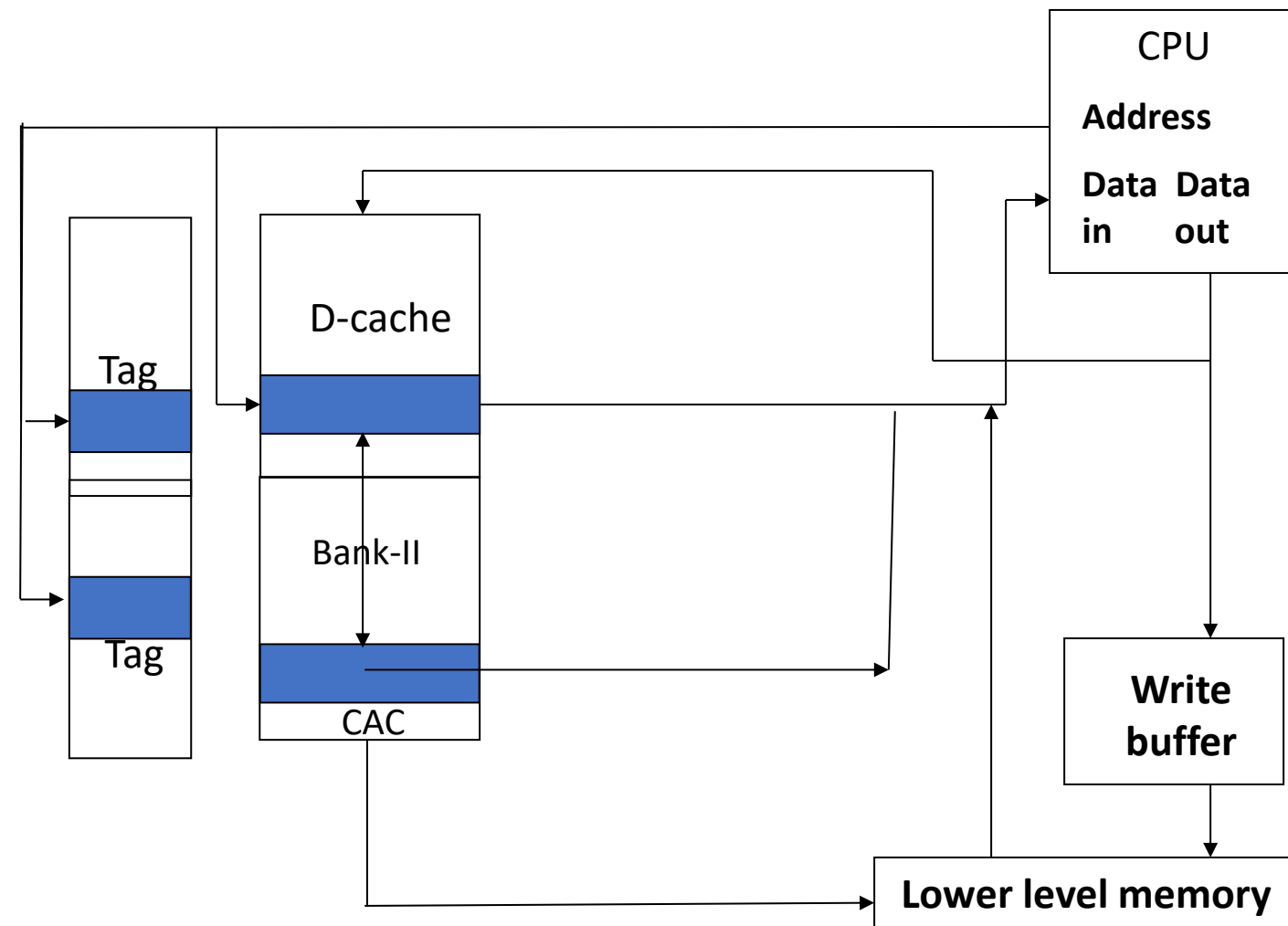


PSEUDO-ASSOCIATIVE (COLUMN-ASSOCIATIVE) CACHES

- Fast hit time of *direct mapped* + lower conflict misses of 2-way SA cache.
- Divide cache: on a miss, check other half of cache; yes? then a pseudo-hit (slow)

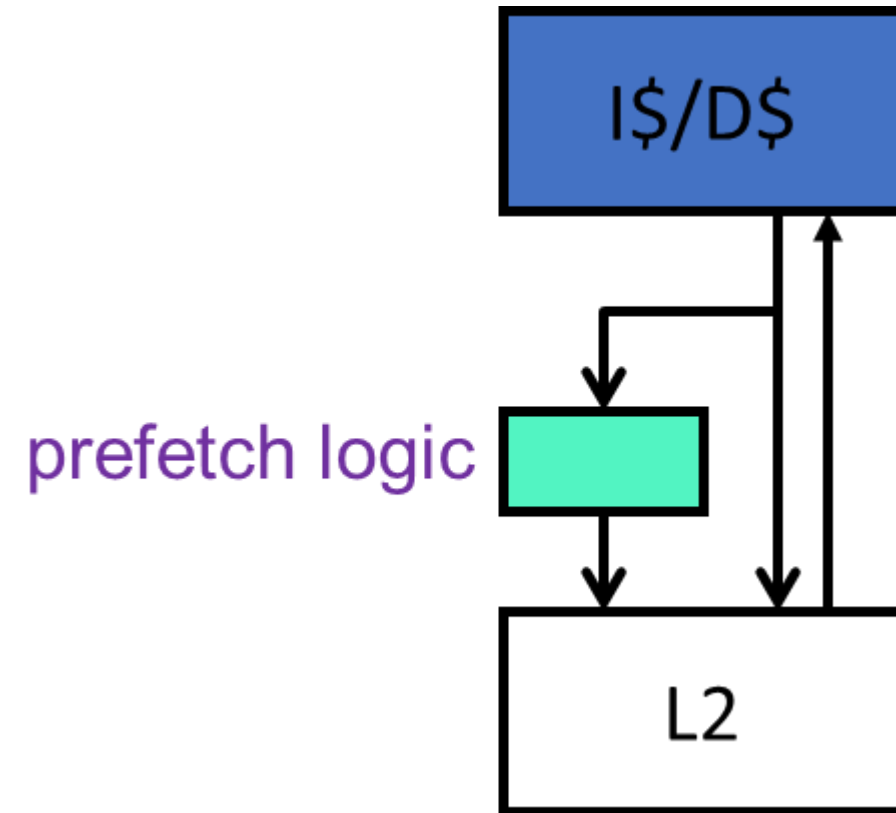


- A simple direct mapped cache, virtually split in two-banks.
- First index to a Set based on 'S' bits. [HIT]
- Upon miss search in alternate bank. [SLOW-HIT]
- Upon miss in alternate bank, go to next level of cache/memory. [MISS]



PREFETCHING

- Bring data into cache **proactively/speculatively**
 - If successful, reduces the number of caches misses
- Key: **anticipate** *upcoming miss addresses* accurately
 - Can be done in either software or hardware.
- Simple hardware prefetching: **next block prefetching**
 - Miss on address **X** → anticipate miss on **X+block-size**
 - + Works for instructions: sequential execution (assumed)
 - + Works for data: arrays
- Table-driven hardware prefetching
 - Use **predictor** to detect strides, common access patterns
- Effectiveness of Prefetching is determined by:
 - **Coverage**: prefetch for as many misses as possible
 - **Timeliness**: initiate prefetches sufficiently in advance
 - **Accuracy**: don't pollute with unnecessary data



SOFTWARE PREFETCHING – ISA SUPPORT?

- Use a special “**prefetch**” instruction
 - Tells the hardware to bring in data
 - Just a hint
- Inserted by programmer or compiler to assist the HW.

Example

```
int tree_add(tree_t* t) {  
    if (t == NULL) return 0;  
    __builtin_prefetch(t->left);  
    __builtin_prefetch(t->right);  
    return t->val + tree_add(t->right) + tree_add(t->left);  
}
```

- Multiple prefetches bring multiple blocks in parallel → More “Memory-level” parallelism (MLP)

Question: How early should we prefetch?

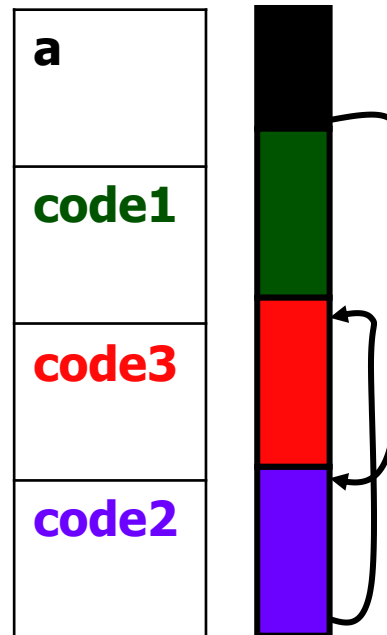
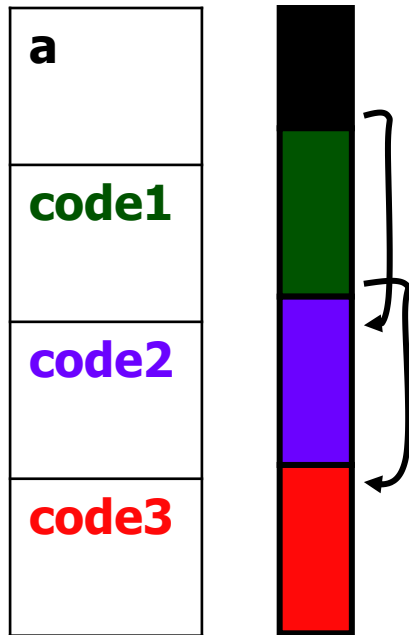
For more details: Read <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html>

CODE OPTIMIZATIONS FOR BETTER PERFORMANCE!

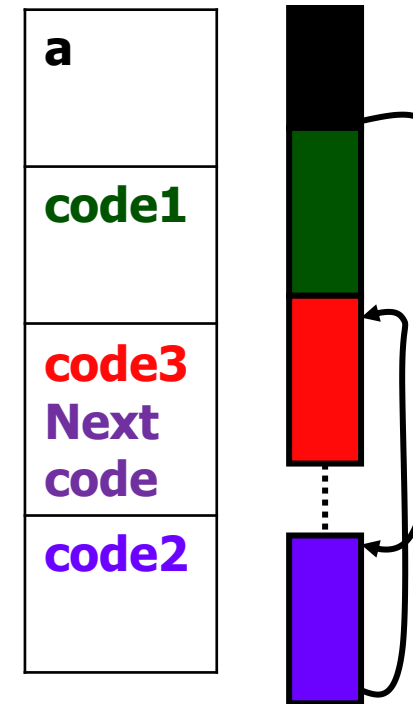
- Avoid hardware changes
- Instructions
 - ***Code Restructuring/Reordering*** to better utilize cache locality.
 - ***Explicit Profiling*** to look at conflicts between groups of instructions
- Data
 - ***Merging Arrays***: improve spatial locality by single array of compound elements vs. 2 arrays
 - ***Loop Interchange***: change nesting of loops to access data in order stored in memory
 - ***Loop Fusion***: Combine 2 independent loops that have same looping construct and some overlapping variables
 - ***Blocking***: Improve temporal locality by accessing “blocks” of data repeatedly vs. going down whole columns or rows.

SOFTWARE RESTRUCTURING: CODE

- Compiler can lay out code for temporal and spatial locality
 - **If (a) { code1; } else { code2; } code3; Nextcode;**
 - But, **code2** case never happens (say, error condition)



- + Better locality
- + Fewer taken branches



- + Better locality for code after **code3**
- + Fewer taken branches

MERGING ARRAYS, LOOP INTERCHANGE AND LOOP FUSION

```
/* Naive: 2 sequential arrays */
```

```
int key[SIZE];
```

```
int val[SIZE];
```

2 misses per access to key & value + potential conflict;

```
/* Optimized: 1 array of structures */
```

```
struct merge {
```

```
int key;
```

```
int val;
```

```
};
```

```
struct merge merged_array[SIZE];
```

Reducing conflicts between key & val; improved spatial locality.

```
/* Naive : Column Major Access*/
```

```
for (j = 0; j < COLS; j = j+1)
```

```
for (i = 0; i < ROWS; i = i+1)
```

```
sum += x[i][j];
```

•Striding through memory every “COL” words;

```
/* Optimized */
```

```
for (i = 0; i < ROWS; i = i+1)
```

```
for (j = 0; j < COLS; j = j+1)
```

```
sum += x[i][j];
```

•Sequential accesses instead of striding through memory every COL words; improved spatial locality

```
/* Naive */
```

```
for (i = 0; i < N; i = i+1)
```

```
for (j = 0; j < N; j = j+1)
```

```
a[i][j] = b[i][j] * c[i][j];
```

```
for (i = 0; i < N; i = i+1)
```

```
for (j = 0; j < N; j = j+1)
```

```
d[i][j] = a[i][j] + c[i][j];
```

2 misses per access to a & c

```
/* Optimized */
```

```
for (i = 0; i < N; i = i+1)
```

```
for (j = 0; j < N; j = j+1)
```

```
{ a[i][j] = b[i][j] * c[i][j];
```

```
d[i][j] = a[i][j] + c[i][j];
```

```
}
```

Now one miss per access; improve temporal locality

SOFTWARE RESTRUCTURING: DATA

```
/* Consider the code below */  
for(k=0; k< NUM_ITERATIONS; ++k)  
    for(i=0; i< NUM_ELEMS; ++i)  
        X[i] = f(X[i]);
```

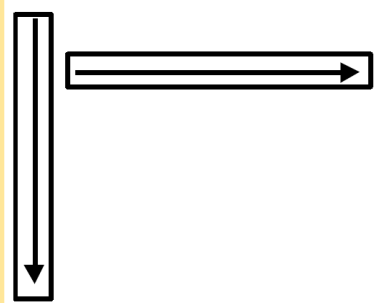
**X[i] is accessed NUM_ITERATION times after accessing all elements of X;
X[NUM_ELEMS] may spill over cache and result in conflict/capacity miss.**

```
/* Optimized: for CACHE_SIZE */  
for(i=0; i<NUM_ELEMS; i+=CACHE_SIZE)  
    for(k=0; k<NUM_ITERATIONS; ++k)  
        for(j=0; j<CACHE_SIZE; ++j)  
            X[i] = f(X[i]);
```

**Slice array into chunks that can fit in a cache; run all iterations;
Better use of temporal locality and spatial locality.**

BLOCKING

```
/* Before */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
    {
        r = 0;
        for (k = 0; k < N; k = k+1)
            r += y[i][k]*z[k][j];
        x[i][j] = r;
    }
```



•Two Inner Loops:

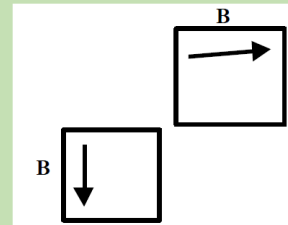
Read all NxN elements of z[]

Read N elements of 1 row of y[] (repeat

Write N elements of 1 row of x[]

•Capacity Misses: function of N & CacheSize

```
/* After BLOCKING :: B = BLOCK_SIZE*/
for (jj = 0; jj < N; jj = jj+B)
    for (kk = 0; kk < N; kk = kk+B)
        for (i = 0; i < N; i = i+1)
            for (j = jj; j < min(jj+B-1,N); j=j+1)
            {
                r = 0;
                for(k=kk; k<min(kk+B-1,N);k =k+1)
                    r += y[i][k]*z[k][j];
                x[i][j] = x[i][j] + r;
            }
```



• Idea: compute on BxB submatrix that fits

B called *Blocking Factor*

Reduce Capacity Misses from $2N^3 + N^2$ to $2N^3/B + N^2$

TECHNIQUES TO REDUCE THE CACHE MISS PENALTY

1. Multi-Level Caches
2. Non-blocking Caches
3. Early restart / Critical Word First (Wrapped Fetch)
4. Sectoring or Sub-block Placement
5. Prioritize Read misses over writes

TECHNIQUES TO REDUCE THE CACHE HIT TIME

1. Small and Simple caches
2. TLB/ Avoid Address Translations – more on this after later.
3. Pipelined writes.

CACHE OPTIMIZATION SUMMARY

Technique	Miss Rate	Miss Penalty	Hit Time	Complexity
Large Cache	+		-	1
Large Block size	+ -	-		0
Higher Associativity	+		-	1
Victim cache	+	+ -	-	2
Pseudo Associate Cache	+	-	-	2
Prefetching	+			3
Prioritize Read misses		+		1
Critical Word first		+		1
Subblock placement		+	+	2
Non blocking Caches		+		3
Multi-Level Caches		+		2
Small and Simple Cache	-		+	0
TLB / Address Translation			+	2
Pipelined Writes			+	1
Multi-ports and Banking				

This slide is inspired by presentations of Randy Katz (UC Berkeley), CS 252.

EIGHT GREAT IDEAS IN COMPUTER ARCHITECTURE

Design for Moore's law

Use Abstraction to Simplify design

Make the Common case fast

Performance Via Parallelism

Performance Via Pipelining

Performance Via Prediction

Hierarchy of Memory

Dependability Via Redundancy